{{PROJECT_TITLE}}

A Project Report

Submitted in partial fulfillment of the requirements for the degree of {{PROGRAM}} (Semester {{SEMESTER}}), under the course {{COURSE}}

Submitted by:

{{AUTHOR_1_NAME}} ({{AUTHOR_1_ROLL_NO}}) {{AUTHOR_2_NAME}} ({{AUTHOR_2_ROLL_NO}}) {{AUTHOR_3_NAME}} ({{AUTHOR_3_ROLL_NO}})

Submitted to:

{{DEPARTMENT}} {{COLLEGE_CAMPUS}} Institute of Science and Technology (IOST) Tribhuvan University Kathmandu, Nepal

{{SUBMISSION_DATE}}

Student's Declaration

We hereby declare that the work presented in this report titled *{{PROJECT_TITLE}}*, submitted in partial fulfillment of the requirements for {{COURSE}} of the {{PROGRAM}} program at {{COLLEGE_CAMPUS}}, {{DEPARTMENT}}, Institute of Science and Technology, Tribhuvan University, is our own original work. It has been carried out under the supervision of {{SUPERVISOR_NAME}}. The material contained in this report has not been submitted, in whole or in part, to this or any other institution for the award of any degree or diploma. All sources of information and prior work used in this report have been duly acknowledged and cited.

Name	Exam / Roll Number	Signature
{{AUTHOR_1_NAME}}	{{AUTHOR_1_EXAM_NO}}	…….
{{AUTHOR_2_NAME}}	{{AUTHOR_2_EXAM_	…….

Name	Exam / Roll Number	Signature
}	NO}}	
{{AUTHOR_3_NAME}	{{AUTHOR_3_EXAM_	…….
}	NO}}	

Date: {{SUBMISSION_DATE}}

Certificate

Supervisor’s Recommendation

I hereby recommend that this report, prepared under my supervision by {{AUTHOR_1_NAME}}, {{AUTHOR_2_NAME}}, and {{AUTHOR_3_NAME}}, titled *{{PROJECT_TITLE}}*, be accepted for evaluation in partial fulfillment of the requirements for {{COURSE}} of the {{PROGRAM}} program. To the best of my knowledge, the work embodied in this report is the candidates’ own and has been carried out satisfactorily.

…….. {{SUPERVISOR_NAME}} Supervisor {{DEPARTMENT}} {{COLLEGE_CAMPUS}}

Letter of Approval

This is to certify that this report titled *{{PROJECT_TITLE}}*, submitted by {{AUTHOR_1_NAME}}, {{AUTHOR_2_NAME}}, and {{AUTHOR_3_NAME}} in partial fulfillment of the requirements for {{COURSE}} of the {{PROGRAM}} program, has been examined and approved in its present form.

……..	……..
{{HOD_COORDINATOR}}	{{SUPERVISOR_NAME}}
Head of Department / Coordinator	Supervisor

{{DEPARTMENT}}	{{DEPARTMENT}}
{{COLLEGE_CAMPUS}}	{{COLLEGE_CAMPUS}}
……..	……..
{{INTERNAL_EXAMINER}}	{{EXTERNAL_EXAMINER}}
Internal Examiner	External Examiner

Date: {{SUBMISSION_DATE}}

Acknowledgement

We express our sincere gratitude to {{DEPARTMENT}}, {{COLLEGE_CAMPUS}}, Institute of Science and Technology, Tribhuvan University, for including project work in the curriculum and for providing the environment in which this project could be undertaken.

We are especially indebted to our supervisor, {{SUPERVISOR_NAME}}, for the continual guidance, technical feedback, and encouragement that shaped this work at every stage. We also thank {{HOD_COORDINATOR}}, Head of Department / Coordinator, and the faculty members of the department for their valuable suggestions during the review sessions.

Finally, we thank our friends and families for their support throughout the development of QForge. Any errors that remain in this report are our own.

{{AUTHOR_1_NAME}} {{AUTHOR_2_NAME}} {{AUTHOR_3_NAME}}

Abstract

Preparing examination question papers by hand is slow, error-prone, and difficult to balance: a setter must cover every unit of a course, respect a fixed distribution of marks and question types, and avoid repeating questions used in recent years, all at once and

under time pressure. QForge is a smart question-paper generation platform built to automate this task while keeping the setter in control. A teacher describes the shape of an examination once as a reusable blueprint of slots, and QForge assembles a valid paper from a bank of approved questions using a deterministic, constraint-based algorithm implemented from scratch. The algorithm combines a greedy selection pass with backtracking repair and is parameterised by a seed, so the same inputs reproduce the same paper and every outcome can be explained. When the question bank is too thin to satisfy a blueprint, an optional, locally hosted artificial-intelligence model proposes candidate questions to top up the bank, while a retrieval-augmented semantic-similarity guard suppresses near-duplicates; the artificial intelligence is supportive only, and the algorithm always decides the final paper. The system is organised as a service-oriented architecture with a Laravel orchestrator, a Python processing service for extraction and generation, and a Vue interface. The implemented system produces valid, balanced, reproducible papers, enforces unit coverage and non-repetition, extracts and imports past papers under review, and closes feasibility gaps through the top-up path, with the full test suite passing. The result is a working hybrid system in which deterministic logic, not the model, owns the examination paper.

Keywords: Question paper generation, Constraint satisfaction, Backtracking, Blueprint, Semantic similarity, Retrieval-augmented generation, Laravel.

Table of Contents

Open in Word and choose "Update Field" to build the Table of Contents.

List of Abbreviations

Abbreviation	Expansion
AI	Artificial Intelligence
AIG	Automatic Item Generation
API	Application Programming Interface
BSc CSIT	Bachelor of Science in Computer Science

Abbreviation	Expansion
	and Information Technology
CRUD	Create, Read, Update, Delete
CSP	Constraint Satisfaction Problem
DFS	Depth-First Search
DOCX	Office Open XML Document format
ER	Entity–Relationship
FK	Foreign Key
HTTP	Hypertext Transfer Protocol
IOST	Institute of Science and Technology
JSON	JavaScript Object Notation
LLM	Large Language Model
LRU	Least Recently Used (cache eviction policy)
OCR	Optical Character Recognition
ORM	Object–Relational Mapping
PDF	Portable Document Format
PK	Primary Key
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
SBERT	Sentence-BERT (sentence embeddings)
SPA	Single-Page Application
TU	Tribhuvan University
UI	User Interface
UML	Unified Modeling Language

List of Figures

List of Tables

Chapter 1: Introduction

1.1 Introduction

Examinations remain the primary instrument by which universities certify what a student has learned, and the question paper is the artefact that carries that judgement. In Tribhuvan University–style courses, a paper is not written freely: it must obey a defined structure — a fixed set of sections, a prescribed number of questions of each type, an exact total of marks — while at the same time drawing across the breadth of the syllabus so that no unit is examined and none is quietly ignored. A well-made paper therefore balances several competing requirements at once, and a teacher setting one must hold all of them in mind together.

In practice this work is done by hand. A teacher assembles each paper individually, choosing questions from memory, from personal files, or from previous years' papers, and then checking by eye that the counts, the marks, the type mix, and the unit spread all come out right. The approach does not scale: it is slow, it is easy to make an arithmetic or coverage error under time pressure, and it offers no systematic guard against repetition, so a question — or a lightly reworded version of it — can reappear across consecutive years without anyone noticing. Nothing about the manual process records *why* a given paper is valid, which makes a paper hard to audit and hard to reproduce, and the effort of setting one from a blank page is repeated in full every examination cycle.

QForge was built to remove that manual burden without removing the teacher's control over the result. The aim of the project was to produce a working system in which a teacher describes the *shape* of an examination once, as a reusable **blueprint**, and the system then assembles a valid, balanced paper from a bank of approved questions automatically. At its centre is a deterministic constraint-based generation algorithm, written from scratch, that treats paper setting as a constraint-satisfaction problem — filling every position, or **slot**, in the paper with a question that honours the hard rules of type, marks, allowed unit, coverage, and non-repetition, while preferring variety where a choice remains. An optional artificial-intelligence component can enlarge the question bank when it is too thin to satisfy a blueprint, but it only ever proposes candidate questions; the algorithm alone decides what appears on the paper. The remainder of this

report sets out the problem in detail, the objectives that followed from it, and the analysis, design, implementation, and testing of the system that met them.

1.2 Problem Statement

Preparing a question paper by hand is a constraint-laden task that becomes error-prone and labour-intensive precisely because several requirements must be satisfied simultaneously and checked against one another. The problem this project addresses is how to generate an examination paper that is *structurally correct, balanced across the syllabus, free of recent repetition, and reproducible*, without depending on a teacher to reconcile all of those requirements manually for every paper.

The requirement has several interacting factors. A paper must satisfy its **structure** — the right number of questions of each type in each section, summing to the intended total marks. It must achieve **unit coverage** — every unit the blueprint admits should be represented, so the examination is not skewed toward whichever topics the setter recalled most readily. It must respect a **mark distribution** and a **question-type mix** fixed in advance. It must avoid **repetition** — questions used in recent papers, and questions from recent real examinations, should not resurface. And it must do all of this under a practical **time** constraint, since a teacher's effort is finite and the setting cycle recurs. Manual setting couples these factors together in one person's head, where a change to satisfy one requirement can silently break another.

The scope of the problem is bounded deliberately. The concern is the generation of TU-style written papers from a curated, institution-owned question bank; it is not automated grading, not proctoring, and not the authoring of question content itself, which remains a human responsibility subject to review. Within those boundaries, however, the problem is general rather than specific to one course: the same structural, coverage, and repetition pressures apply to any subject whose examinations are organised into units and marks, so a solution framed at the level of blueprints and constraints applies across a department's offerings rather than to a single paper.

The justification for solving it is direct. Setting papers manually consumes hours of skilled staff time each cycle for a result that is neither auditable nor guaranteed correct, and the absence of any systematic repetition guard is a genuine fairness and integrity risk when students can obtain and memorise recent papers. A system that produces balanced

papers automatically, proves each paper correct against an explicit checklist, and enforces non-repetition therefore addresses a real, recurring, and non-trivial cost — it does more than describe the difficulty of paper setting; it frames it as a problem to be solved reliably and repeatably.

1.3 Objectives

The objectives of the project, each of which the delivered system meets and each of which maps to a factor in the problem above, were:

- To build a **constraint-based, blueprint-driven generator** that produces a valid, balanced paper from a reusable blueprint — every slot filled, the required per-section counts and total marks satisfied — or, when no such paper exists, a precise account of what is missing.
- To **enforce unit coverage, per-unit maximum caps, and cross-paper non-repetition** as hard rules of generation, so that structure, balance, and freedom from recent repetition are guaranteed rather than checked after the fact.
- To make generation **reproducible and explainable** — deterministic given a recorded **seed**, and accompanied by a per-constraint pass/fail result so that any paper can be reproduced exactly and any failure is stated as named **missing slots**.
- To **ingest past-paper and syllabus documents** through an extraction pipeline (digital text with optical-character-recognition fallback and heuristic parsing) into an **admin-reviewed** question bank, so the bank can be grown from existing material without unverified questions entering it.
- To provide an **optional AI top-up** that expands the bank with grounded candidate questions when a blueprint is otherwise infeasible — guarded against semantic duplicates and kept strictly supportive of the algorithm — and to **export** finished papers to PDF and DOCX.

1.4 Scope and Limitation

The system covers the full path from a curated question bank to a finished paper. In scope are the authoring of blueprints; deterministic generation of papers from them with coverage, cap, and repetition enforcement; management of the question bank; ingestion of past-paper and syllabus PDFs into that bank through admin-reviewed extraction; an

optional AI top-up that enlarges the bank when a blueprint cannot otherwise be satisfied; and export of saved papers to PDF and DOCX. Two operating roles are supported — an Admin who stewards the catalog and the bank, and a Teacher who authors blueprints and papers.

The limitations are stated honestly and reflect deliberate design choices rather than gaps. The artificial-intelligence component is **supportive only**: it drafts candidate question text, which the system then stamps with type, marks, and unit and subjects to the same constraints as any other question — the algorithm, not the model, owns every selection, so the AI cannot cause an invalid paper. Document extraction is heuristic by design and therefore **requires admin review**: parsed candidates enter the bank only after a human approves them, which trades full automation for control over what becomes examinable. Semantic duplicate detection is threshold-based and so catches typical paraphrases but is not an absolute guarantee. Finally, the system assumes a **single-institution** deployment with TU-style, unit-and-marks-structured papers and an English-language question corpus; multi-institution operation and other examination formats are outside the present scope.

1.5 Development Methodology

The project followed an **iterative, milestone-first** methodology in which each milestone delivered an independently demonstrable slice of the system, working end-to-end against the live backend rather than as an isolated component. Crucially, the sequencing was **algorithm-first**: because the constraint-based generation engine is the academic centrepiece and the highest-risk part of the build, it was implemented and proven early — against seeded, in-memory data — before the messier document-processing and AI pipelines were added on top of a foundation already known to be correct.

The build proceeded through six principal milestones. The first established the domain foundation — the schema, the CRUD interfaces, and role-based access. The second delivered the generation engine itself. The third added the paper lifecycle, export, and cross-paper repetition control. The fourth built the PDF extraction pipeline and, in a follow-on, syllabus import. The fifth added AI bank expansion, and the sixth added retrieval-augmented grounding with the semantic-duplicate guard. Each milestone left the system in a working, demoable state, and the algorithm’s correctness was pinned by a

growing unit-test suite throughout, so that later additions could not silently regress the engine that underpins the whole system.

1.6 Report Organization

The remainder of this report is organised as follows. **Chapter 2** reviews the fundamental concepts the system rests on — constraint satisfaction, greedy heuristics and backtracking, document extraction, semantic similarity, and AI-assisted generation — and surveys prior work on automated paper and question generation, positioning QForge's hybrid design against it. **Chapter 3** presents the system analysis: the functional and non-functional requirements, a feasibility assessment, and the object-oriented models of the problem domain. **Chapter 4** presents the system design, refining those models to the level at which they were implemented, adding the component, deployment, and persistence views, and giving a full treatment of the generation algorithm — the centre of gravity of the report. **Chapter 5** describes the implementation of the real modules and the testing strategy, including the unit and system test suites and an analysis of the results against the objectives. **Chapter 6** concludes by restating how the objectives were met and setting out honest recommendations for future work.

Chapter 2: Background Study and Literature Review

2.1 Background Study

QForge draws together ideas from several distinct fields — combinatorial search, educational assessment, document digitisation, and natural-language processing — and combines them into a single working system. This section examines each of the underlying concepts, but always in the specific role it plays inside QForge, rather than as an abstract topic. The intent is not to define these ideas in general terms but to make clear *why* each one is needed and *where* the built system relies on it.

2.1.1 Constraint Satisfaction, Greedy Selection, and Backtracking

At its core, assembling an examination paper from a fixed template is a constraint-satisfaction problem: a set of positions must each be given a value, subject to rules that the finished assignment as a whole must respect. QForge casts every position in a paper — a **slot** of a particular type and mark value — as a variable, the pool of approved questions that legally fit it as that variable's domain, and the blueprint's rules (correct type and marks, an allowed unit, coverage of every unit, and no repetition) as the constraints the completed paper must satisfy. Framing the task this way is what allows the system to report, for any produced paper, exactly which constraints held, and for any failure, exactly which requirement could not be met.

Two complementary strategies from the study of such problems drive the generation engine. The first is a greedy heuristic: rather than searching exhaustively, the engine walks the slots in order and commits, at each step, to the locally most attractive candidate — the question that covers a still-uncovered unit and has been used least often. Greedy methods of this kind are attractive because a single forward pass is fast and, on the common case, sufficient [7]. Their known weakness is that a choice which looks best in isolation can foreclose a valid completion later on; greedy selection offers no guarantee of finding a solution even when one exists. QForge therefore pairs it with a second strategy — backtracking — which supplies precisely the guarantee the greedy pass lacks. Backtracking explores the space of assignments systematically, extending a partial solution one slot at a time and retracting the most recent choice whenever a slot can no longer be filled or the coverage requirement can no longer be met [6]. Because it revisits

earlier decisions, it will locate a valid paper whenever one is reachable within its search budget, recovering exactly the cases the myopic greedy pass mishandles. The engine runs the fast greedy fill first and invokes the bounded backtracking search only as a repair pass when validation fails, so the system pays for the more expensive search only when it is actually required.

2.1.2 Blueprints and Test Specifications in Assessment

The notion that an examination should be built against an explicit specification of its shape — how many questions of each kind, carrying what marks, drawn from which parts of the syllabus — predates any software and is well established in assessment practice as a test blueprint or table of specifications, a device for ensuring that a test samples the intended content in the intended proportions rather than reflecting whatever the setter happened to recall {{CITATION NEEDED}}. QForge makes this specification a first-class, reusable object: a teacher authors a **blueprint** once, describing the paper's sections, the type and mark value of each slot, the units the paper may draw from, and the repetition rules it must observe. Every later generation is an attempt to realise that blueprint from the current question bank. Capturing the specification explicitly is what turns the balance of a paper from a matter of the setter's memory into a property the system can enforce and verify, and it is the artefact that makes generation repeatable across examination cycles.

2.1.3 Optical Character Recognition and PDF Text Extraction

To seed and grow its question bank from a department's existing material, QForge must recover machine- readable text from PDF documents of two very different kinds. A born-digital PDF already carries an embedded text layer, and its characters can be read directly; a scanned or photocopied paper is only a bitmap image and carries no such layer, so its text must be reconstructed by optical character recognition — the conversion of an image of printed text into the characters it depicts. QForge uses the Tesseract engine for this reconstruction [4], rasterising a page and passing it through OCR only when the page is judged to be a scan. The decision is made per page rather than per document, because real examination papers routinely mix a digitally typeset cover sheet with photocopied question pages, and a single scanned page can even carry a misleading text layer baked in by the scanner's own low-quality OCR. Recognising that recovered text is inherently noisier than embedded text, the extraction pipeline treats every OCR-derived candidate as

provisional and routes it through human review before it can ever enter the bank the generator draws upon.

2.1.4 Embeddings and Semantic Similarity for Duplicate Detection

Deciding whether two questions are effectively the same cannot be done by comparing their characters: “Define a binary tree” and “What is a binary tree?” are the same question in different words, while a literal text match would treat them as unrelated. QForge resolves meaning rather than spelling by means of embeddings — representations that map a piece of text to a fixed-length vector of numbers positioned so that texts of similar meaning land close together, regardless of the particular words they use. The closeness of two such vectors is measured by cosine similarity, the cosine of the angle between them, which yields a score that rises toward one as two texts converge in meaning. This family of techniques was advanced substantially by sentence-level embedding models that produce directly comparable vectors for whole sentences, making semantic comparison efficient enough to apply at scale [5]. QForge puts this single operation to work in several guises: flagging a freshly extracted question that paraphrases one already in the bank, discarding AI-written candidates that merely reword existing ones, and steering the generator away from placing two questions that mean the same thing on one paper. In each case the comparison is a similarity score against stored vectors, and in each case that score informs a decision without ever overriding the deterministic rules that govern the paper itself.

2.1.5 Large-Language-Model-Assisted Question Generation

When a blueprint demands more questions of some type, mark value, and unit than the bank can supply, QForge can call a locally hosted large language model to draft fresh candidate questions and close the shortfall. Automatically producing assessment items in this way is an established line of research: systematic surveys of automatic question generation document a field that has moved from rule- and template-based methods toward neural generation [2], and the broader theory of automatic item generation frames how assessment items can be produced systematically rather than authored one by one [3]. A recognised hazard of unconstrained language-model generation is that the model, lacking knowledge of the specific syllabus, will produce fluent but off-topic or fabricated content. QForge mitigates this by grounding every generation prompt in the relevant syllabus text and exemplar questions retrieved from its own corpus, so the model writes

about the intended material rather than inventing it. Crucially, the generated text is treated only as candidate wording: the system stamps the question's type, marks, and unit from the blueprint slot rather than trusting the model, and the deterministic algorithm alone decides which questions reach the paper. The AI is therefore supportive, never authoritative — it enlarges the pool of raw material, and nothing more.

2.2 Literature Review

Automated construction of examination and test papers has been studied for some time, and the proposed approaches can be grouped into a small number of recurring strategies. Reviewing them by strategy — rather than paper by paper — makes clear both what each family achieves and where it falls short, and thereby exposes the gap that QForge's hybrid design is intended to fill.

The simplest strategy selects questions at random from a bank until a paper's requirements appear to be met. Its appeal is that it is trivial to implement and introduces variety between successive papers, but the guarantees it can offer are correspondingly weak: random selection cannot ensure that every required topic is covered or that mark totals come out exactly, and when it fails to produce an acceptable paper it offers no account of why. It is best understood as a baseline against which more disciplined methods are measured [1].

A second and more heavily studied family treats paper composition as an optimisation problem and applies metaheuristic search — most prominently genetic algorithms and ant-colony optimisation — to navigate the very large space of possible question combinations toward one that scores well against a weighted objective. Work in this vein has modelled the multiple, competing demands on a paper (its coverage, difficulty distribution, and total marks) as terms in a fitness function and evolved or foraged for a combination that balances them [1], with related studies pursuing the same goal through refined genetic operators {{CITATION NEEDED}}. These methods handle large banks and multi-objective trade-offs well, but they carry two costs relevant here. Because they optimise a weighted aggregate, a strong score on some objectives can compensate for a violated requirement on another, so a hard constraint such as full unit coverage is not strictly guaranteed; and the stochastic search that gives them their reach also makes a given result harder to reproduce and to explain to an examiner.

A third family stays closer to the constraint-satisfaction framing and uses systematic search — backtracking and related constraint-solving techniques — to construct a paper that provably meets every hard rule, an approach whose foundations lie in the classical treatment of constraint satisfaction [6]. Its strength is exactly the guarantee the metaheuristic methods relax: if a valid paper exists, a complete search will find it, and the reasoning is transparent. Its weakness is cost, since an unguided search over a large bank can become expensive, which is why practical systems temper it with heuristics or bounds.

A fourth and more recent direction shifts the focus from selecting existing questions to generating new ones, using automatic item generation and, latterly, large language models to author assessment content directly [2], [3]. This line addresses a problem the selection-based families cannot — a bank too thin to satisfy any request — but it introduces concerns of its own around the correctness, relevance, and originality of generated items, and it does not by itself decide how those items are assembled into a balanced paper.

Alongside these algorithmic strands sit integrated question-bank and paper-generation systems that package storage, selection, and paper assembly into a single administrative tool {{CITATION NEEDED}}. Such systems demonstrate the practical value of the workflow but tend to treat the selection step as a supporting feature rather than the object of study, and they seldom foreground reproducibility or an explicit account of constraint satisfaction.

Across these approaches a gap emerges. The random and metaheuristic families buy variety at the cost of guaranteed constraint satisfaction and reproducibility; the systematic-search family buys those guarantees but addresses only selection from an adequate bank; and the generative family can enlarge a thin bank but cannot on its own assemble a valid paper. No single prior approach delivers all of determinism, guaranteed constraint satisfaction, an explainable outcome, and a remedy for an insufficient bank at once. QForge is positioned in precisely this space. It makes a deterministic greedy-plus-backtracking engine the sole authority over paper composition — securing guaranteed constraint satisfaction, reproducibility for a given seed, and a per-constraint explanation of every result — while relegating language-model generation to a strictly supporting role that replenishes the bank without ever deciding the paper. This division of labour, in

which rule-based search owns correctness and AI merely supplies raw material behind it, is the hybrid design the remainder of this report develops.

Chapter 3: System Analysis

System analysis fixes *what* QForge must do and models the problem it solves, before any implementation detail is settled. QForge was built as a hybrid question-paper generator: a deterministic, blueprint-driven engine owns the final paper, while an optional artificial-intelligence component only supplements the question bank when the engine cannot otherwise satisfy a request. This chapter records the requirements that shaped the system, the feasibility of building it under the project's constraints, and the object-oriented models of the problem domain. Because QForge is a completed system, the analysis is presented as realised rather than proposed, and every model is drawn from the delivered code rather than from an early sketch. The solution-level refinements — the service class structure, the architecture, and the relational persistence schema — are deferred to the design chapter, where they belong.

3.1 System Analysis

3.1.1 Requirement Analysis

Requirement analysis separates the behaviour the system exposes to its users (functional requirements) from the qualities that behaviour must exhibit (non-functional requirements). QForge's requirements were derived from the two operating roles it serves and from the end-to-end workflow of turning a syllabus and a bank of past questions into a balanced examination paper.

Functional Requirements

QForge recognises exactly two human roles, stored on each user account and derived on the server at login: the **Admin**, who acts as the content steward, and the **Teacher**, who authors papers. A third, secondary actor — the **System (the Python processing service)** — participates only when Laravel delegates document processing or text generation to it; it never initiates work and is never reached directly by a user. The frontend communicates only with the Laravel application, and Laravel in turn orchestrates the Python service, so the artificial-intelligence contribution remains supportive rather than authoritative: the deterministic engine always controls the paper that is produced.

The Admin curates everything the generator later draws upon. An Admin provisions user accounts and assigns roles, maintains the catalog of subjects and their units, and owns the

question bank with full create, read, update, and delete access. Because much of the bank originates from real examinations, the Admin also uploads past-paper and syllabus documents, and works the resulting review queue — approving or rejecting the candidates the extraction pipeline proposes, and confirming a parsed syllabus into subjects and units before it is written to the catalog. There is no public registration; every account is created by an Admin.

The Teacher consumes that curated material to produce papers. A Teacher builds a **blueprint** — the specification of the paper's sections, per-section question counts and marks, allowed units, unit coverage rules, per-unit maximums, and repetition exclusions — and then requests generation from it. When the approved bank is too thin to satisfy a blueprint, the Teacher may trigger an AI **top-up** that expands the bank with newly generated questions and then regenerate. A produced paper can be saved, edited, exported to PDF or DOCX, and reviewed later through history and analytics screens. Blueprints and papers are private to their owner: a Teacher sees only their own.

These capabilities and the actors that exercise them are summarised in the use case diagram of Figure 3.1, which also shows where one use case draws in another — extraction is included by every document upload, AI question generation is included by the bank top-up, and the top-up itself extends generation as an optional recovery path.

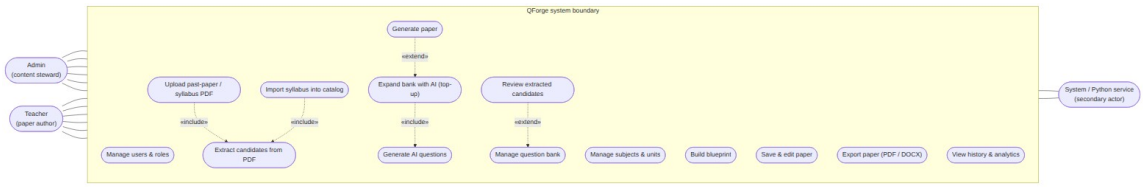

Figure 3.1: Use case diagram — actors and use cases of QForge.

The principal use cases are described in Table 3.1, which records each actor, the use case, and its behaviour.

Table 3.1: Use case descriptions.

Actor	Use case	Description
Admin	Manage users & roles	Create, update, and remove accounts and assign the admin/teacher role; the last remaining admin cannot be

Actor	Use case	Description
		demoted or self-deleted.
Admin	Manage subjects & units	Maintain the catalog of subjects (keyed by public code) and the ordered units within each.
Admin	Manage question bank	Full CRUD over questions, including type, marks, unit tagging, and status.
Admin	Upload past-paper / syllabus PDF	Submit a document for asynchronous processing; includes *extract candidates from PDF*.
Admin	Review extracted candidates	Approve or reject parsed candidates (individually or in bulk); approval extends the bank.
Admin	Import syllabus into catalog	Confirm a parsed syllabus proposal, creating the subject and any missing units.
Teacher	Build blueprint	Define sections, counts, marks, allowed units, coverage rules, per-unit maximums, and exclusions.
Teacher	Generate paper	Run the deterministic engine against a blueprint to obtain a paper preview or a precise shortfall.
Teacher	Expand bank with AI (top-up)	On an infeasible blueprint, queue AI generation of the missing questions; includes

Actor	Use case	Description
		generate AI questions.
Teacher	Save & edit paper	Persist a generated preview and adjust it.
Teacher	Export paper	Render a saved paper to PDF or DOCX.
Teacher	View history & analytics	Browse previously generated papers and usage aggregates.
System	Extract candidates from PDF	Parse an uploaded document into question candidates, with OCR fallback for scanned pages.
System	Generate AI questions	Produce grounded candidate questions for named missing slots via the local language model.

Non-Functional Requirements

The qualities QForge had to exhibit follow directly from its purpose as an examination tool that must be trustworthy, defensible, and deployable inside a single institution.

Determinism and reproducibility. Because an examination paper must be auditable and repeatable, the generation engine is a pure function of the question bank and a single integer **seed**. A null seed reproduces the original identifier-ordered paper, which keeps the test suite stable; a supplied seed yields a different but equally valid paper, so a regenerate action and two teachers working at once no longer collide on one frozen result. Any given seed always reproduces the same paper, which is the property that makes the output defensible.

Performance and responsiveness. Generation itself runs synchronously in memory as plain server-side logic, so a Teacher receives a paper preview without waiting on external services. Every heavy or slow task — PDF text extraction, optical character recognition,

AI question generation, and embedding — is instead dispatched to a background queue and polled for completion, so a single slow document never blocks the interface.

Security. Access is authenticated with token-based bearer credentials issued at login, and every protected route runs behind role middleware that admits only the permitted role and refuses everyone else. There is no public signup, roles are decided server-side rather than trusted from the client, and blueprints and papers are owner-scoped so one Teacher cannot read another's work.

Maintainability. The system keeps strict service boundaries: Laravel is the orchestrator that owns the database, the business logic, and the generation algorithm; the Python service is a stateless processor with no database of its own; and the frontend is presentation only. Keeping the language model behind a swappable provider and the paper renderers behind one shared view-model localises future change.

Portability. The whole system — application, worker, database, cache, vector index, language model, and frontend — is described as a set of containers, so a reviewer or an institution can stand the platform up reproducibly on their own hardware without per-machine configuration.

3.1.2 Feasibility Analysis

Feasibility was assessed against the resources and constraints of an undergraduate project delivered by a small team on institution-scale hardware, rather than in the abstract.

Technical feasibility. Every component of QForge is built on a mature, well-documented technology with a large support base: a PHP application framework for the orchestrator and API, a reactive JavaScript framework for the interface, a Python web framework for the processing service, a relational database for the source of truth, an in-memory store with a queue dashboard for background work, a vector index for semantic retrieval, and a self-hostable language-model runtime. Critically, the academic centrepiece — the constraint-based generation engine — is implemented in plain application code with no dependency on an external constraint solver, so its behaviour is fully owned, inspectable, and unit-testable. This removed the main technical risk: correctness of the algorithm did not hinge on a third-party black box.

Operational feasibility. The design mirrors how examination papers are actually prepared in a department. Content stewardship (the catalog and the bank) is separated

from paper authoring, matching the real division between those who maintain question stores and those who set papers. Extraction and AI generation never write directly into the usable bank: a human reviews extracted candidates before approval, so the automation assists staff without displacing their judgement. The workflow therefore slots into an existing process rather than demanding a new one.

Economic feasibility. QForge is composed entirely of open-source software, so there are no licensing costs. The decision that most affects running cost is that language-model inference is served by a **self-hosted** runtime rather than a commercial API: AI question generation and embedding incur no per-call charge and send no institutional content to an outside vendor. The recurring cost is therefore limited to the institution's own hardware and electricity, which makes routine and repeated use economically sustainable.

Schedule feasibility. The project was sequenced algorithm-first and delivered as a series of independently demonstrable milestones, so that the highest-risk, highest-value component — the generation engine — was proven early against seeded data before the messier document and AI pipelines were added. The milestone order was: the domain foundation and CRUD (M1); the generation engine (M2); the paper lifecycle, export, and repetition control (M3); the PDF extraction pipeline (M4) and syllabus import (M4.1); AI bank expansion (M5); and semantic retrieval with the duplicate guard (M6). This ordering is shown as a schedule in Figure 3.2. The milestone sequence is real; the calendar dates on each bar are left as placeholders to be completed with the project's actual dates before the final report is produced.

> **[FIGURE PLACEHOLDER — Figure3.2.]** diagram source did not render (contains unresolved `{{PLACEHOLDER}}` tokens); insert the finished figure here before submission.

Figure 3.2: Project schedule — the algorithm-first milestone sequence (calendar dates are placeholders pending the project's actual timeline).

3.1.3 Analysis (Object-Oriented)

The problem domain is modelled with the object-oriented approach used consistently across the analysis and design of QForge. This section presents the problem-level models: the classes and their relationships, one concrete instance snapshot, the lifecycle of the key stateful entities, the two principal interaction sequences, and the control flow of the

generation algorithm. Each model was reconciled against the delivered code before it was drawn; the refined service classes, the deployment topology, and the relational schema are treated in the design chapter.

Class Diagram

The domain revolves around a small set of collaborating classes. A `Subject` owns an ordered set of `Unit` objects and scopes a bank of `Question` objects; a `Question` is filed under one primary `Unit` but may be tagged with several units at once, so a question that genuinely spans topics remains eligible whenever any of its units is allowed. A `Blueprint`, owned by a user and targeting a subject, holds its specification in a structured definition. A `Paper` is generated from a blueprint for a subject and is composed of `PaperQuestion` entries, each a snapshot of the chosen question at the moment of generation — recording its section, display number, marks, unit label, and whether it came from AI. A `DocumentUpload` records an uploaded file and the progress of its asynchronous processing. These classes and their multiplicities are shown in Figure 3.3.

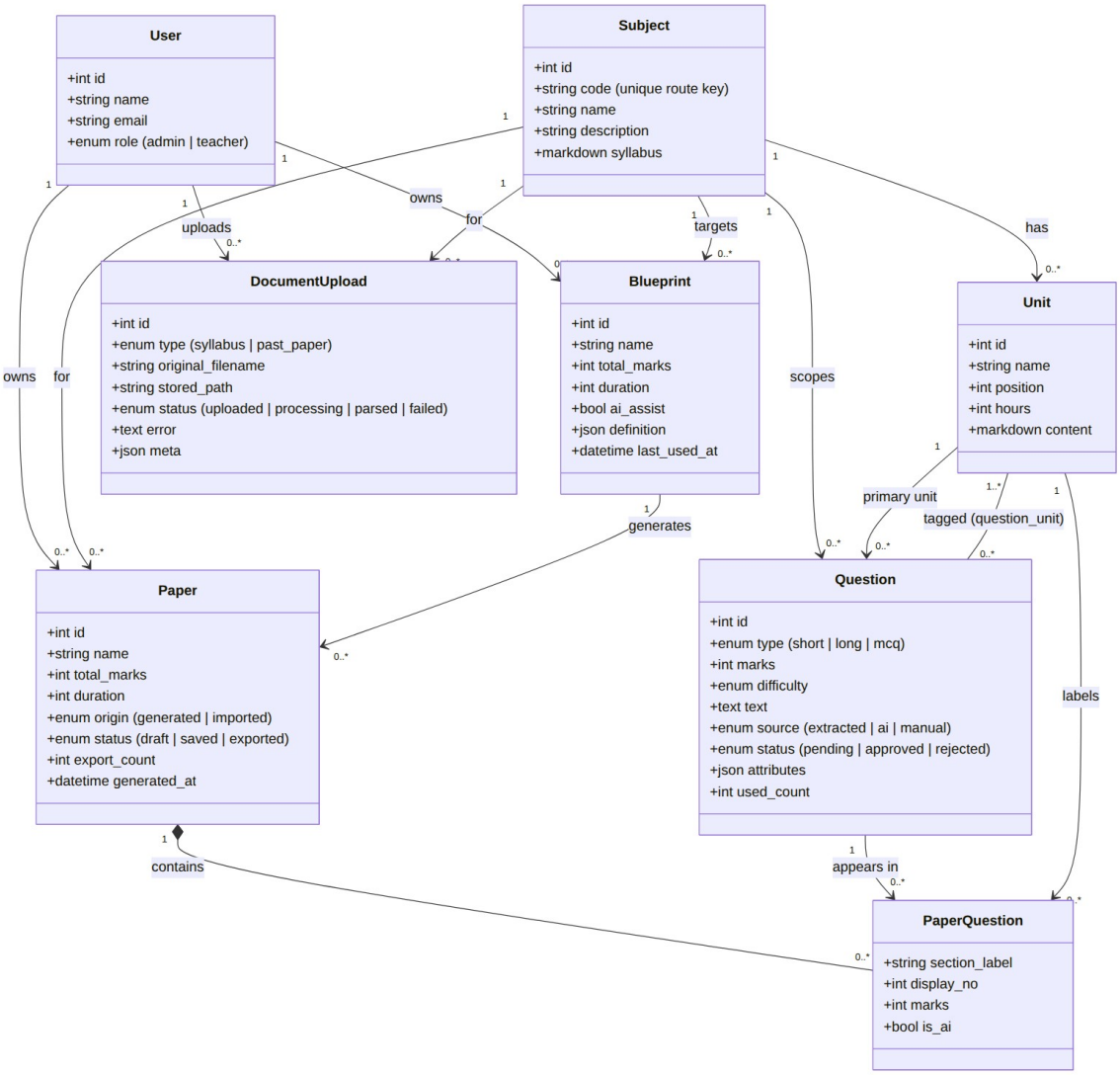

Figure 3.3: Analysis-level class diagram of the QForge problem domain.

Object Diagram

To make the class model concrete, Figure 3.4 shows a single runtime snapshot. A blueprint for the subject `CS301` is compiled into an ordered set of **slots**; the generation engine fills those slots with specific approved `Question` instances from the bank; and the resulting saved `Paper` records each choice as a `PaperQuestion` snapshot. The slot is a transient object produced during generation rather than a stored entity, which is why it appears here as an instance without a persistent counterpart in the class diagram.

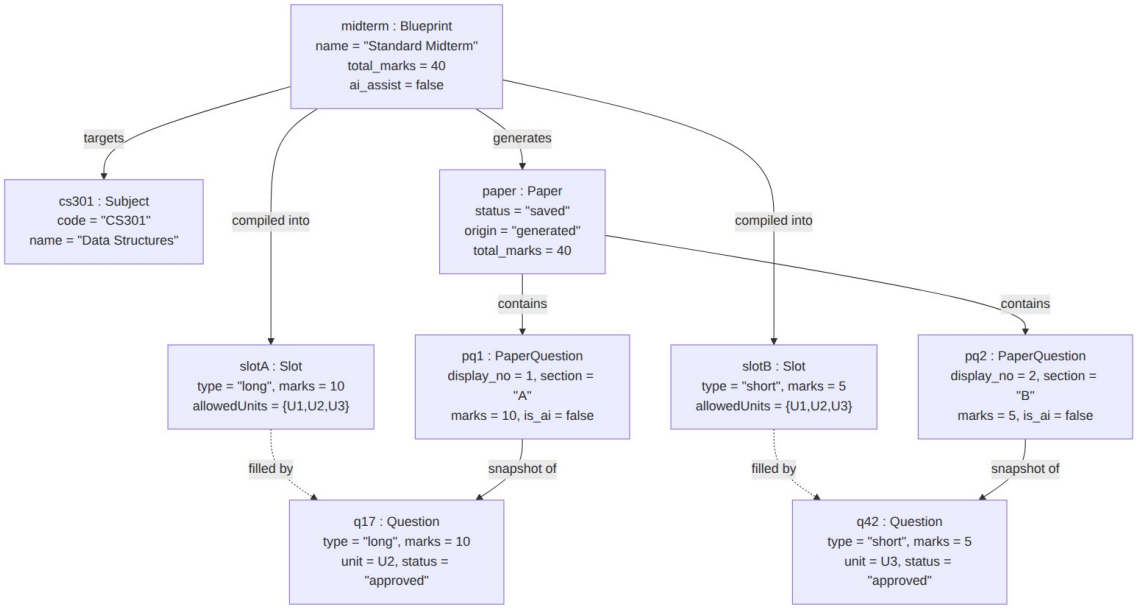

Figure 3.4: Object diagram — a worked instance snapshot of one generated paper.

State Diagrams

Three entities carry a lifecycle worth modelling explicitly. The first is the `Question`, whose `status` governs whether the engine may use it. As Figure 3.5 shows, an extracted or manually created question enters as *pending* and becomes usable only when an Admin approves it, or is set aside when rejected. Questions produced by the AI top-up are the exception: they are stored directly as *approved*, because a pending question is invisible to the candidate filter and could therefore never close the shortfall it was generated to fix. Only approved questions are eligible for generation.

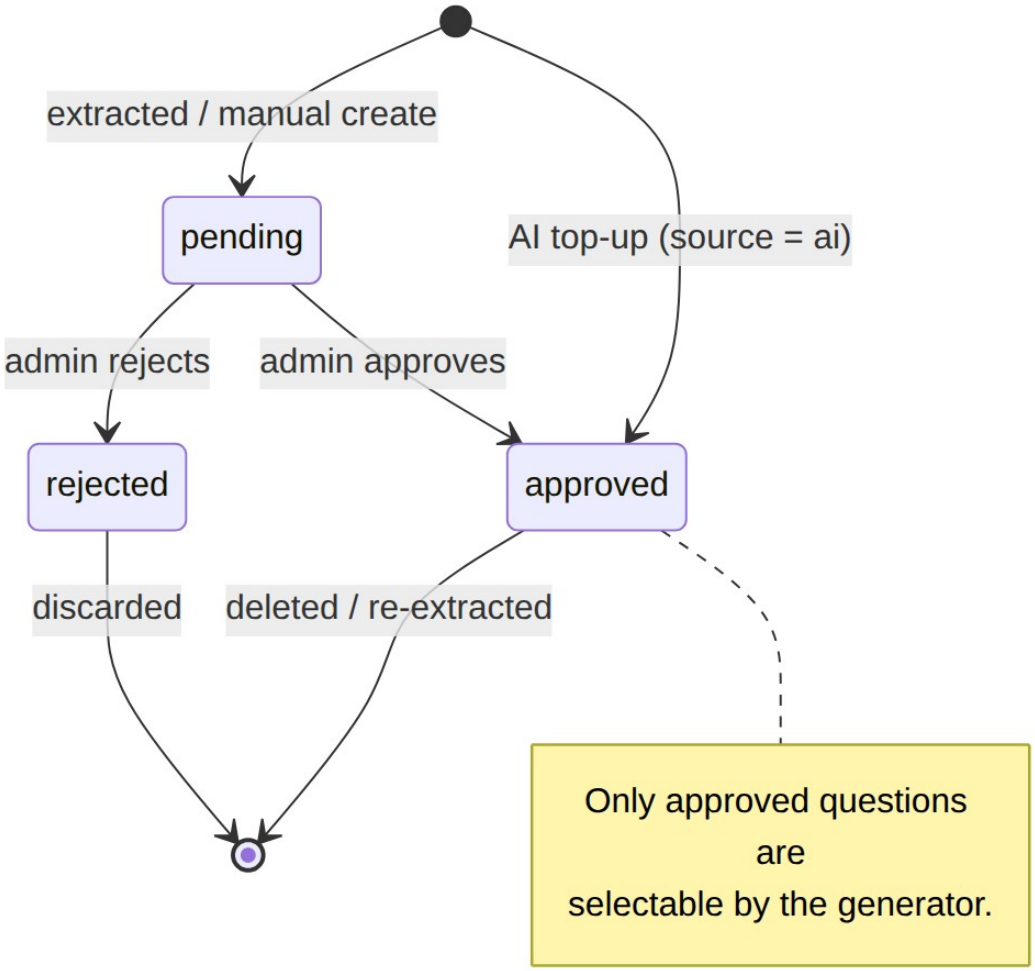

Figure 3.5: State diagram — Question approval lifecycle.

The second is the `Paper`. As Figure 3.6 shows, generation produces an unsaved **preview** that holds its seed but is persisted to nothing; regenerating simply draws a fresh seed and produces another preview. Only an explicit save re-runs the engine deterministically from the seed and persists the paper, after which it may be exported. This preview-first lifecycle keeps discarded attempts out of a teacher's history. (An earlier *draft* state, retained as a legacy value, is no longer produced by the current flow.)

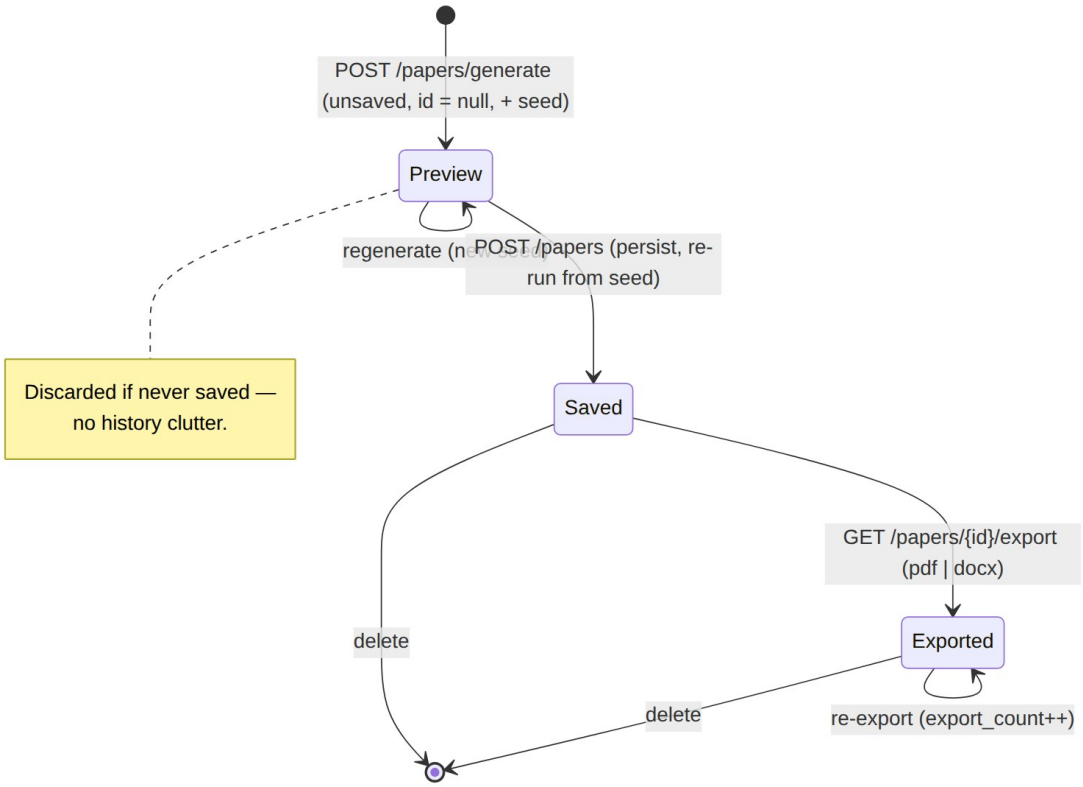

Figure 3.6: State diagram — Paper preview/save/export lifecycle.

The third is the `document_uploads` record, which tracks an asynchronous extraction. As Figure 3.7 shows, an upload begins as *uploaded*, moves to *processing* when the background job picks it up, and finishes either as *parsed*, when the processing service returns candidates, or as *failed*. Both end states are terminal.

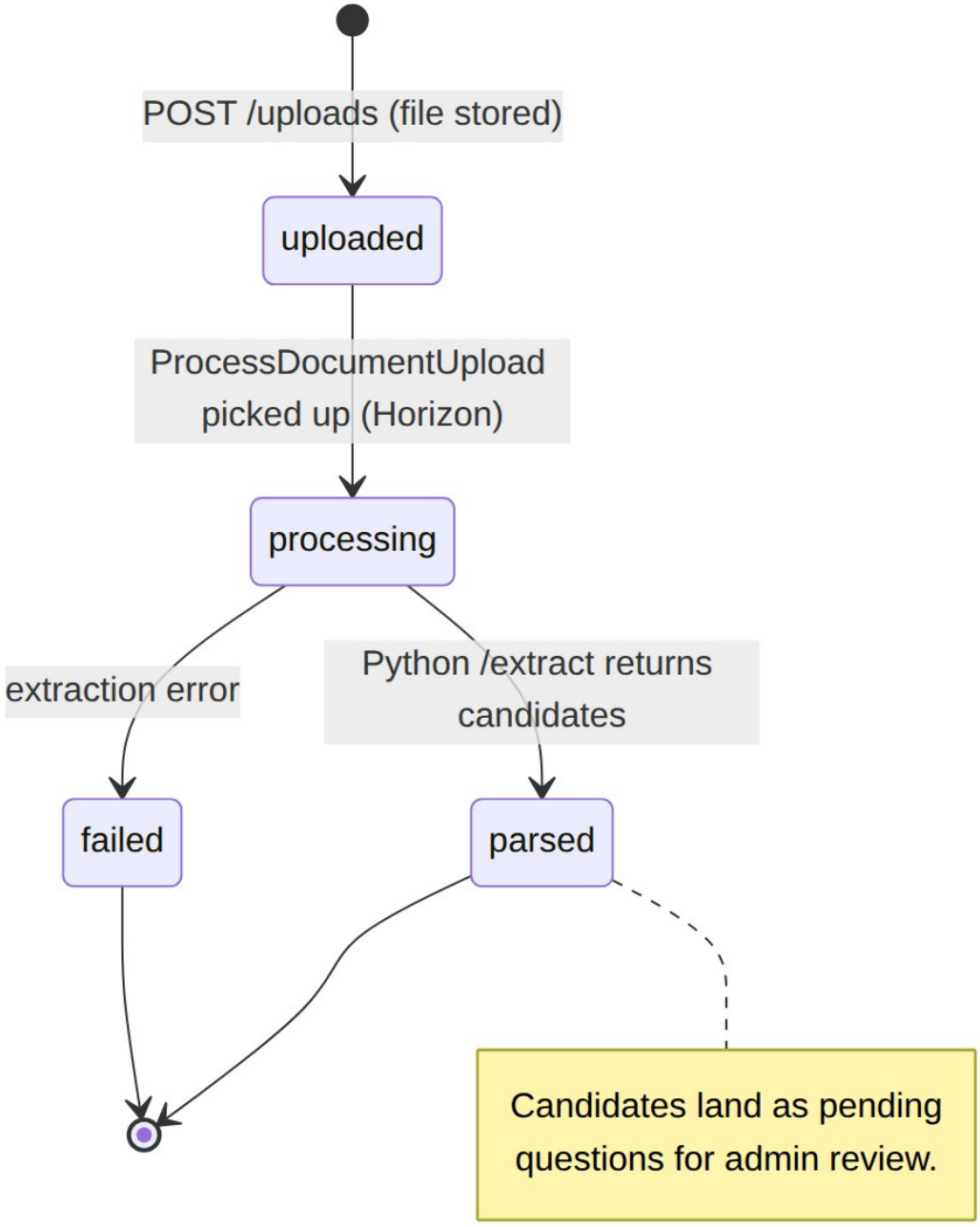

Figure 3.7: State diagram — document_uploads processing lifecycle.

Sequence Diagrams

The two interactions that define the system are generating a paper and extracting questions from a document. Figure 3.8 traces generation: the Teacher requests a paper from a blueprint, the compiler turns the blueprint into ordered slots, the engine filters and greedily selects a **candidate** for each slot, validation checks the assembled paper, and the result is returned as an unsaved preview together with its seed. If the greedy pass and

validation cannot produce a valid paper, backtracking attempts alternative selections; if the bank is genuinely too thin, the response instead names the exact **missing slots**. Persistence occurs only on a subsequent, explicit save.

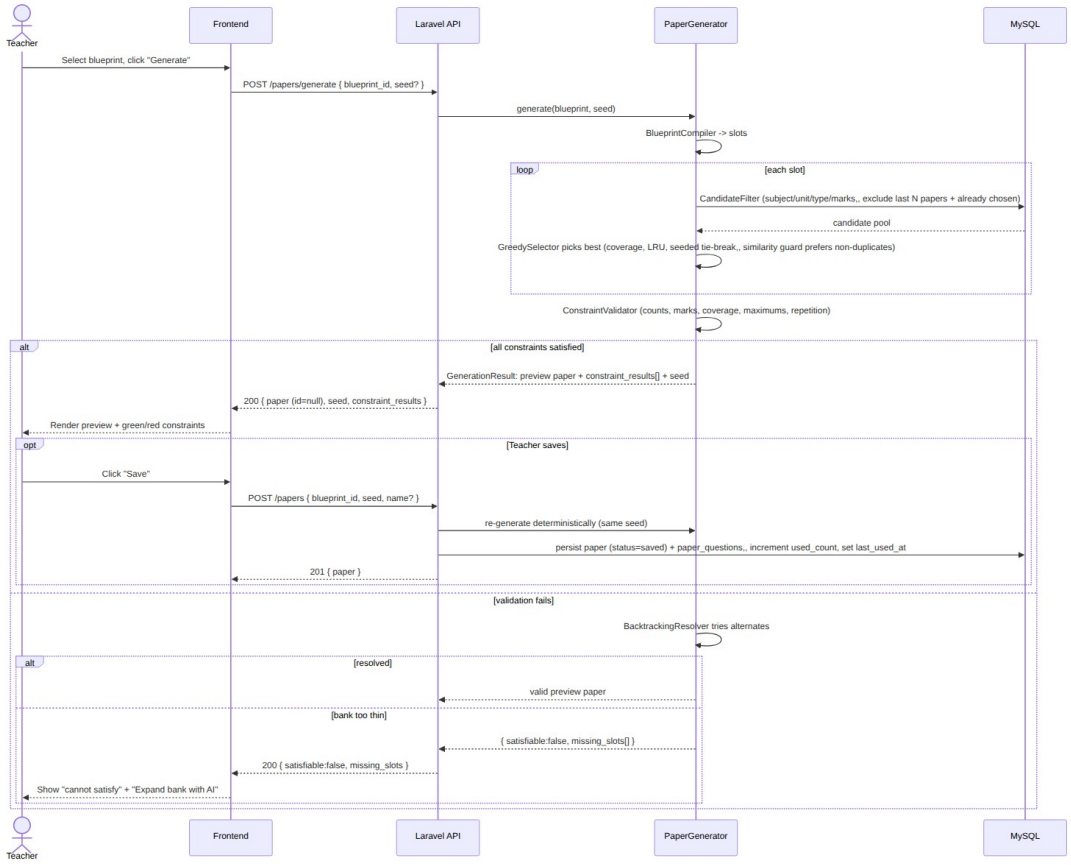

Figure 3.8: Sequence diagram — generating a paper (deterministic, synchronous, preview-first).

Figure 3.9 traces extraction, which is asynchronous. The Admin uploads a document; the API records it and dispatches a background job, returning immediately while the frontend polls for progress. The job calls the Python service, which extracts text — falling back to optical character recognition on pages that carry little or no digital text — and parses the content into candidates. Those candidates are stored as pending questions, and the Admin later approves them into the bank through the review queue.

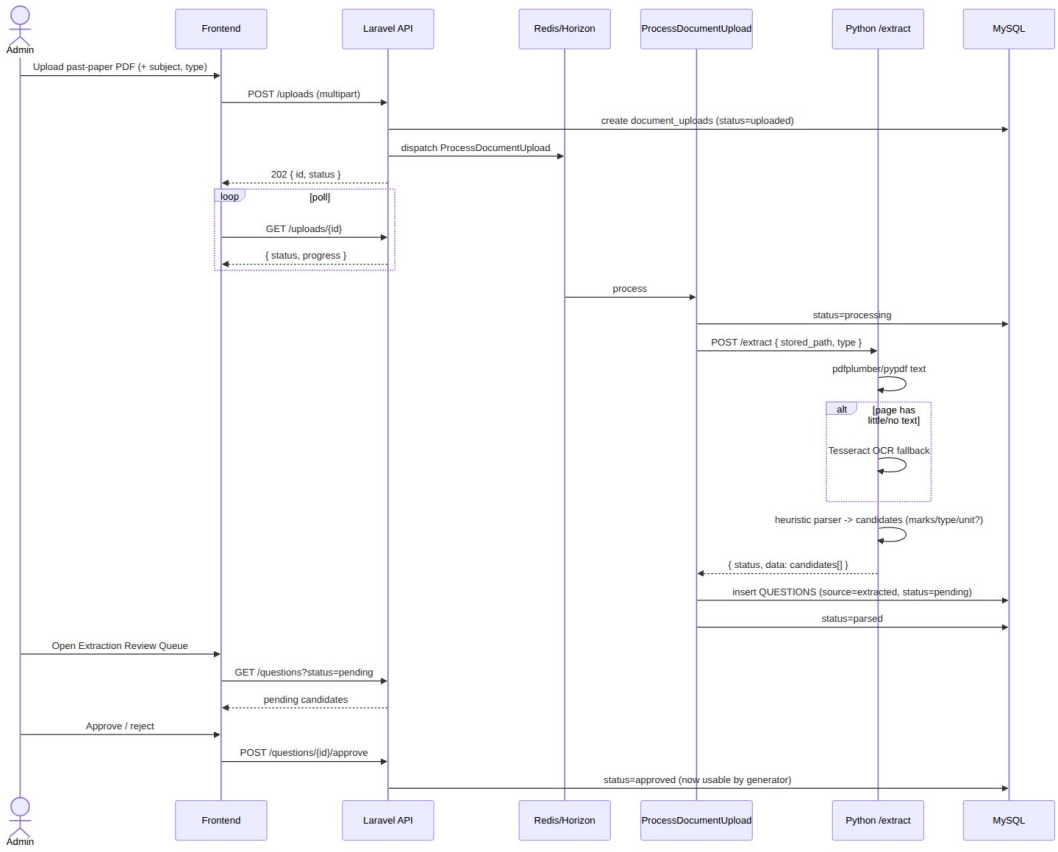

Figure 3.9: Sequence diagram — extracting question candidates from an uploaded document.

Activity Diagram

Finally, Figure 3.10 models the control flow of the generation algorithm itself. The blueprint is compiled into ordered slots; a greedy pass then walks the slots, filtering the bank for each and selecting a candidate. Once every slot has been attempted, the constraint validator checks the whole paper. If all constraints hold, the run succeeds and returns a preview. If not, a bounded backtracking search tries alternative assignments; only when that too fails does the engine compute and return the precise **missing slots** describing what the bank lacks. This is the analysis-level view of the control flow; the detailed treatment of the algorithm’s phases, its seed-parameterised determinism, and the **similarity guard** is the subject of the design chapter.

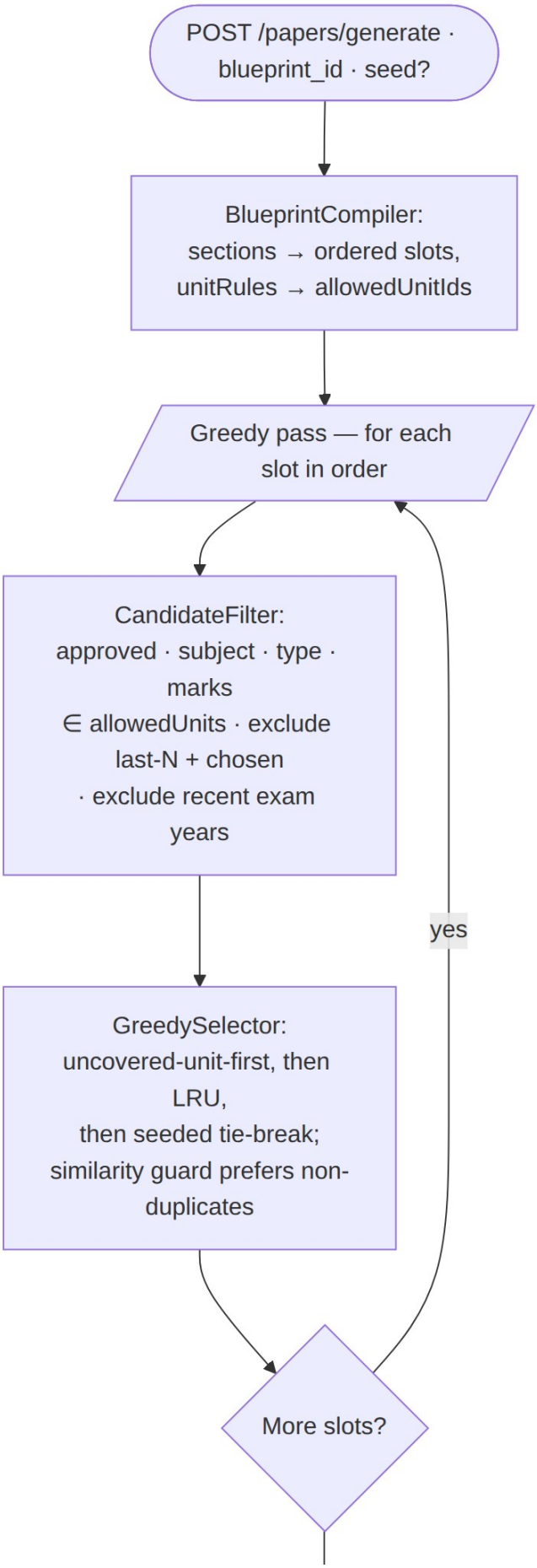
POST /papers/generate · blueprint_id · seed?
BlueprintCompiler: sections → ordered slots, unitRules → allowedUnitIds
Greedy pass — for each slot in order
CandidateFilter: approved · subject · type · marks ∈ allowedUnits · exclude last-N + chosen · exclude recent exam years
GreedySelector: uncovered-unit-first, then LRU, then seeded tie-break; similarity guard prefers non-duplicates
More slots?
yes

Figure 3.10: Activity diagram — control flow of the paper-generation algorithm.

Chapter 4: System Design

System design turns the *what* established in the previous chapter into the *how*: the same object-oriented models are refined to the level at which they were actually implemented, and the architecture, deployment topology, and persistence schema that support them are made explicit. The chapter has two parts. Section 4.1 refines the analysis models — the class, object, state, sequence, and activity diagrams — and adds the solution-level views that had no place in analysis: the component architecture, the container deployment, and the relational schema treated as a persistence view beneath the object design. Section 4.2 is the centre of gravity of the whole report: a full treatment of the paper-generation algorithm, its five phases, its seed-parameterised determinism, its soft semantic-duplicate guard, and its complexity. Throughout, one principle is kept in front: the deterministic engine owns the final paper, and the artificial-intelligence component is supportive only — it may enlarge the question bank, but it never selects, orders, or approves what appears on a paper.

Every model in this chapter was reconciled against the delivered code before it was drawn, and the class, method, and table names are reproduced exactly as they appear in the source.

4.1 Design

The analysis chapter modelled the *problem*; this section refines those models into the *solution* as built. The refinement is a continuous progression rather than a fresh start — the domain classes of the analysis carry through unchanged, and the design adds the internal service structure that realises the behaviour, the runtime collaborations between those services, the concrete deployment, and the persistence schema.

Refined Class Diagram

The single most important refinement is the internal structure of the generation engine. In analysis the engine was one conceptual responsibility; in design it is a small collaboration of single-responsibility classes under the `App\Services\PaperGeneration` namespace, coordinated by a facade. The facade, `PaperGenerator`, composes five collaborators — `BlueprintCompiler`, `CandidateFilter`, `GreedySelector`, `ConstraintValidator`, and `BacktrackingResolver` — and consults a

`SimilarityGuard` during the greedy pass only. The guard is an interface with two implementations: the default `NullSimilarityGuard`, a no-op that keeps the engine a pure function of the bank, and `VectorSimilarityGuard`, which compares embedding vectors held in Qdrant. Candidate ordering ends in a `TieBreaker` whose seeded key is the only source of run-to-run variation. The compiler produces a `CompiledBlueprint` value object carrying the ordered `Slot` list, and the facade returns a `GenerationResult` carrying either the selections and an all-pass `ConstraintResult` list or a partial result plus a `MissingSlot` list. These service classes and their relationships are shown in Figure 4.1.

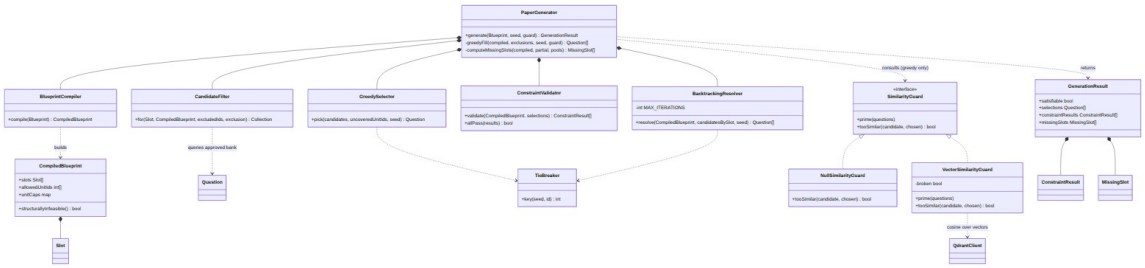

Figure 4.1: Refined class diagram — the PaperGeneration service classes and their collaborations.

The design deliberately keeps the engine free of persistence. `PaperGenerator` reads the question bank but writes nothing; turning a preview into a saved paper is the controller’s responsibility. Isolating selection logic from database writes is what makes the engine unit-testable as a pure function and is the reason the whole algorithm can be exercised deterministically against seeded fixtures.

Refined Object Diagram

Carrying the instance-level view through to design, Figure 4.2 shows the objects that collaborate during a single run of the refined engine. The `PaperGenerator` compiles the blueprint into one `CompiledBlueprint` holding the ordered slots, the allowed units, and the per-unit caps; it consults a `VectorSimilarityGuard` during the greedy pass; and it delegates the per-slot decision to a `GreedySelector` that returns concrete approved `Question` instances. The run terminates in one `GenerationResult` whose `selections` map each slot index to the chosen question. Compared with the analysis-level object snapshot, this diagram exposes the engine’s own runtime objects rather than only the persisted domain entities.

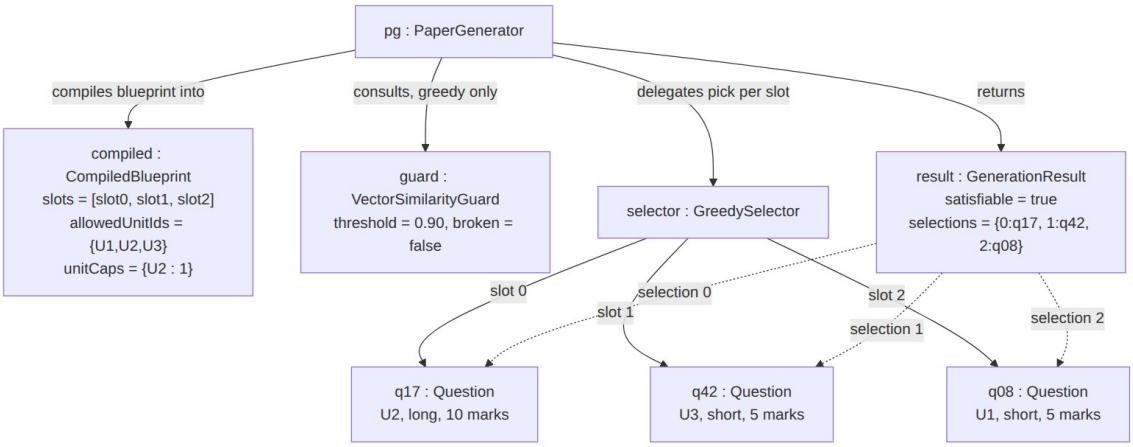

Figure 4.2: Refined object diagram — the collaborating objects of one generation run.

Refined State Diagram

The paper lifecycle is refined to expose the two possible outcomes of a preview and the artificial-intelligence recovery path that connects an infeasible preview back to a feasible one. As Figure 4.3 shows, a generate request enters the *Previewing* state and resolves either to a *FeasiblePreview* (every constraint satisfied) or an *InfeasiblePreview* (the engine returned a shortfall). From an infeasible preview the Teacher may regenerate with a fresh seed, or invoke the AI **top-up**, which enlarges the bank with newly approved questions and returns to *Previewing* so the engine can run again over the larger bank. Only a feasible preview may be saved, after which the paper may be exported. This refinement makes the supportive role of AI explicit at the level of the state machine: the top-up transition feeds the bank and loops back through generation; it never produces a saved paper directly.

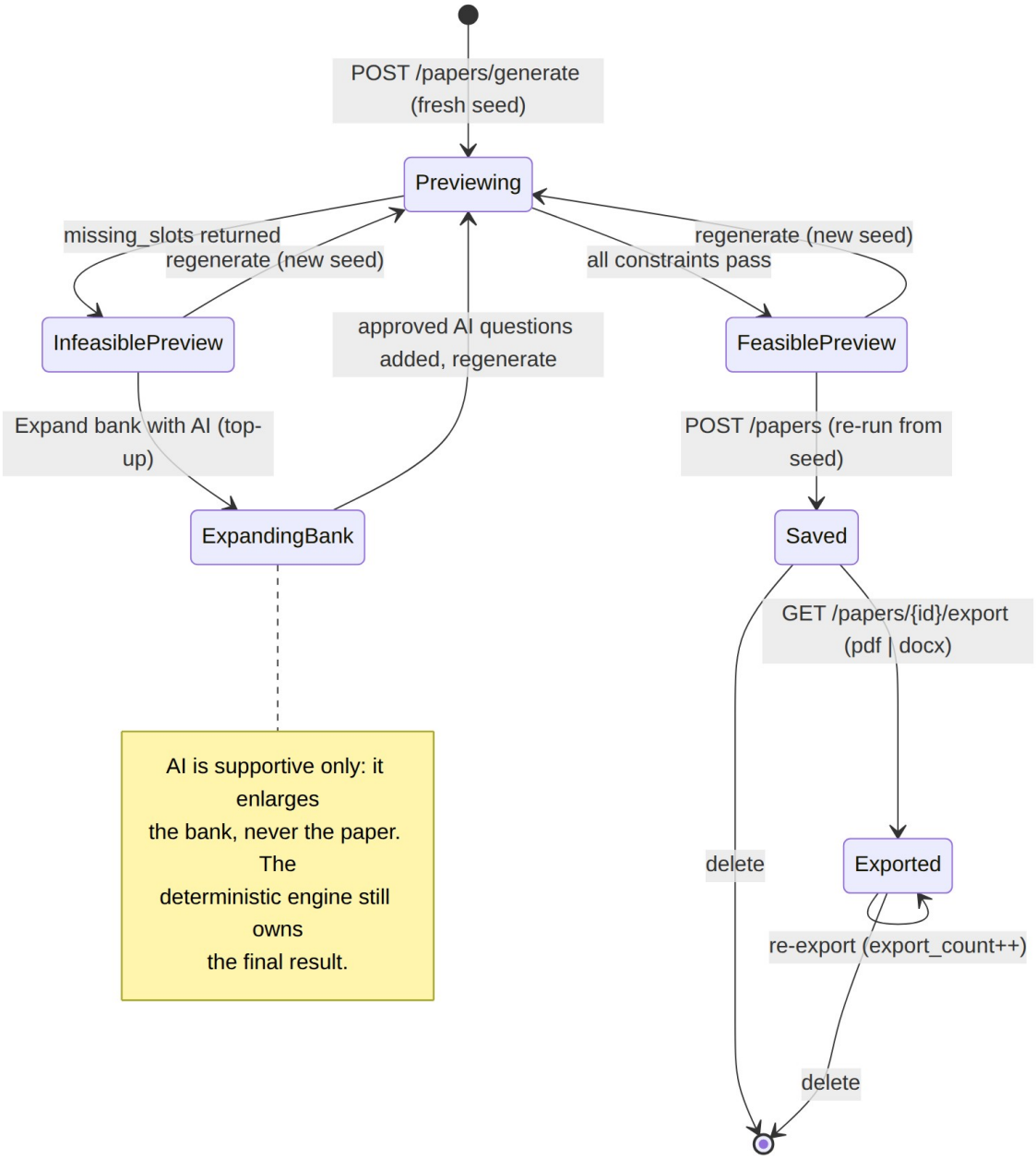

Figure 4.3: Refined state diagram — generation outcomes and the AI top-up recovery path.

Refined Sequence Diagram

Figure 4.4 refines the generation interaction to include the two elements absent from the analysis sequence: the semantic similarity guard consulted inside the greedy loop, and the AI top-up path taken when the bank is too thin. During each slot's turn the engine primes the guard with the candidate pool and asks whether a

candidate is a near-duplicate of one already placed; the guard retrieves the relevant question vectors from Qdrant and answers by cosine similarity, failing open — reporting "not similar" — whenever the vector index is unavailable, so a soft preference can never break generation. When neither the greedy pass nor backtracking can produce a valid paper, the response names the missing slots, and the Teacher may dispatch the ExpandQuestionBank job. That job calls the Python /generate-questions endpoint with a retrieval-grounded prompt, the local language model drafts candidate questions, and they are inserted as approved AI questions before the Teacher regenerates. The correct communication flow — frontend to Laravel to Python and back — is preserved throughout; the frontend never contacts Python directly.

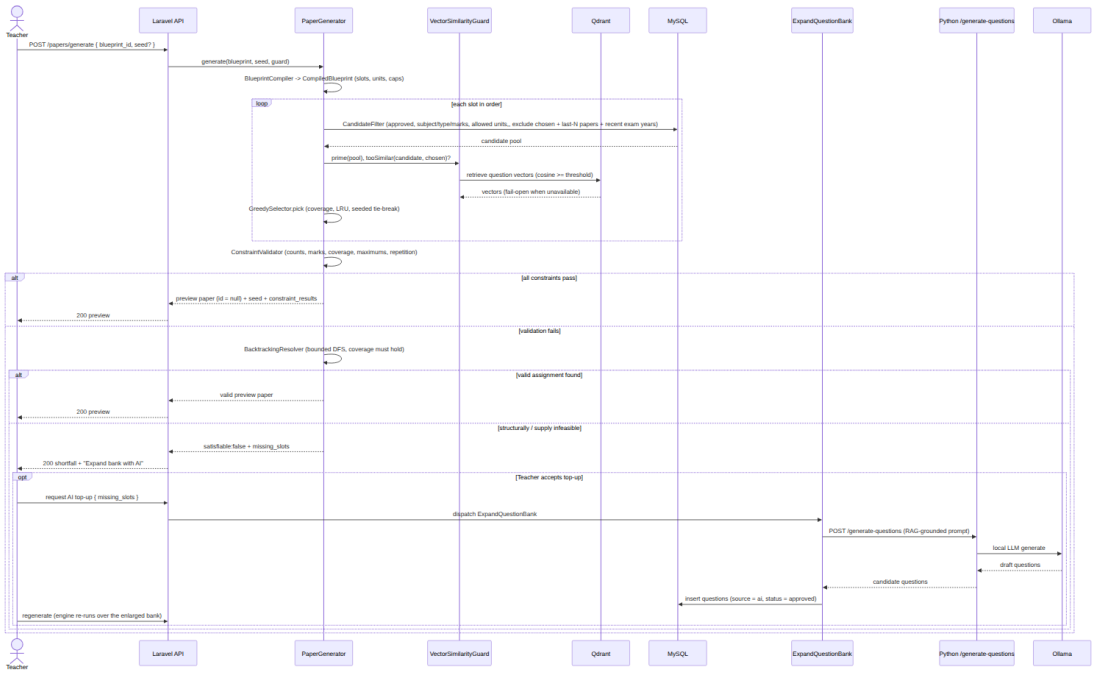

Figure 4.4: Refined sequence diagram — generation with the RAG similarity guard and the AI top-up path.

Refined Activity Diagram

The control flow of the algorithm is refined from the analysis-level sketch into the full five-phase process actually implemented, shown in Figure 4.5. The refinements are the structural-infeasibility short-circuit that skips the backtracking search when no assignment can exist, the cap-busting rejection and soft duplicate preference inside the candidate step, the seeded tie-break at the end of every ordering, and the preview-then-

save lifecycle. This diagram is the visual companion to the phase-by-phase prose of Section 4.2; the two were reconciled against each other and against the source before drawing.

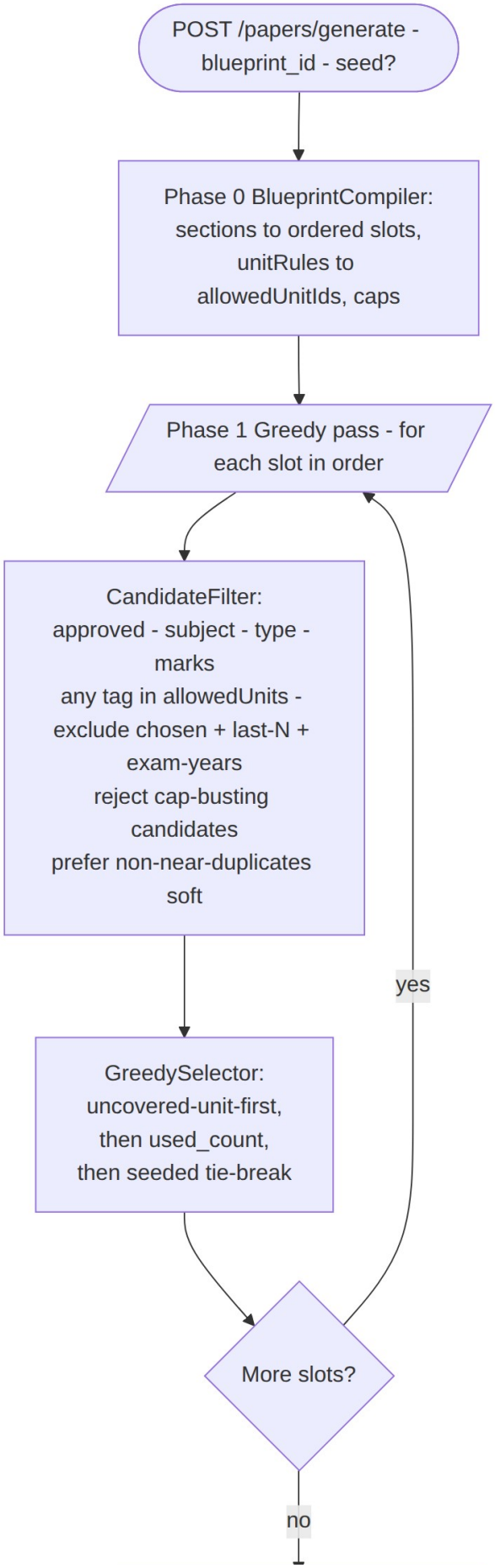
POST /papers/generate - blueprint_id - seed?
Phase 0 BlueprintCompiler: sections to ordered slots, unitRules to allowedUnitIds, caps
Phase 1 Greedy pass - for each slot in order
CandidateFilter: approved - subject - type - marks
any tag in allowedUnits - exclude chosen + last-N + exam-years
reject cap-busting candidates
prefer non-near-duplicates soft
GreedySelector: uncovered-unit-first, then used_count, then seeded tie-break
More slots?
yes
no

Figure 4.5: Refined activity diagram — the five phases of generation with the structural short-circuit and preview/save lifecycle.

Component Diagram

At the architectural level QForge is a service-oriented system with Laravel as the sole orchestrator. Figure 4.6 shows the components and the direction of every dependency. The Vue single-page application is presentation only and speaks exclusively to the Laravel REST API over `/api`, authenticated by Sanctum tokens and gated by role middleware. Laravel owns the MySQL source of truth, runs the pure-PHP `PaperGeneration` service in-process, and dispatches heavy work to Redis-backed queued jobs. The Python FastAPI service is a stateless processor exposing `/extract`, `/generate-questions`, and `/embed`; it holds no business logic and never touches the main database. Laravel reaches Python only from jobs and through a configured base URL, never a hard-coded host, and Python reaches the local Ollama runtime for language-model and embedding inference. The RAG services within Laravel — including the `VectorSimilarityGuard` that the engine consults — read and write the Qdrant vector index directly over REST, exactly as they use Redis.

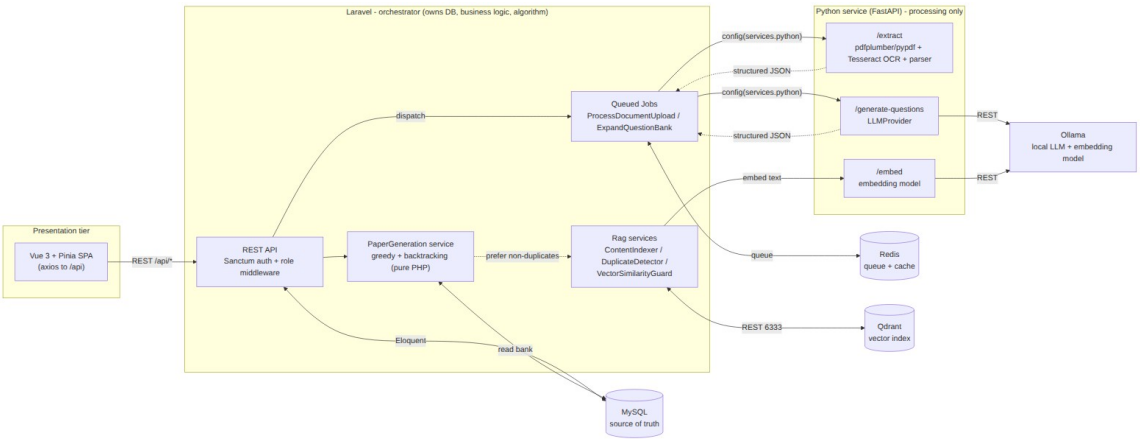

Figure 4.6: Component diagram — Laravel orchestrator, Python processor, Vue presentation, and the backing stores.

Deployment Diagram

The whole system is described as a Docker Compose topology so it can be reproduced on any host without per-machine configuration. Figure 4.7 shows the containers, the single bridge network they share, the host ports each publishes, and the named volumes that persist state across restarts. An Nginx container fronts both the PHP-FPM application and

the Vite development server; a separate worker container runs the Horizon queue processor under supervisord; and MySQL, Redis, Qdrant, the FastAPI service, and Ollama each run in their own container on the same `local` network. Three named volumes give durability to the data that must survive a rebuild: the MySQL data directory, the Ollama model cache, and the Qdrant index storage. Because the vector index is rebuildable from the relational source of truth, its volume is a performance convenience rather than an authoritative store.

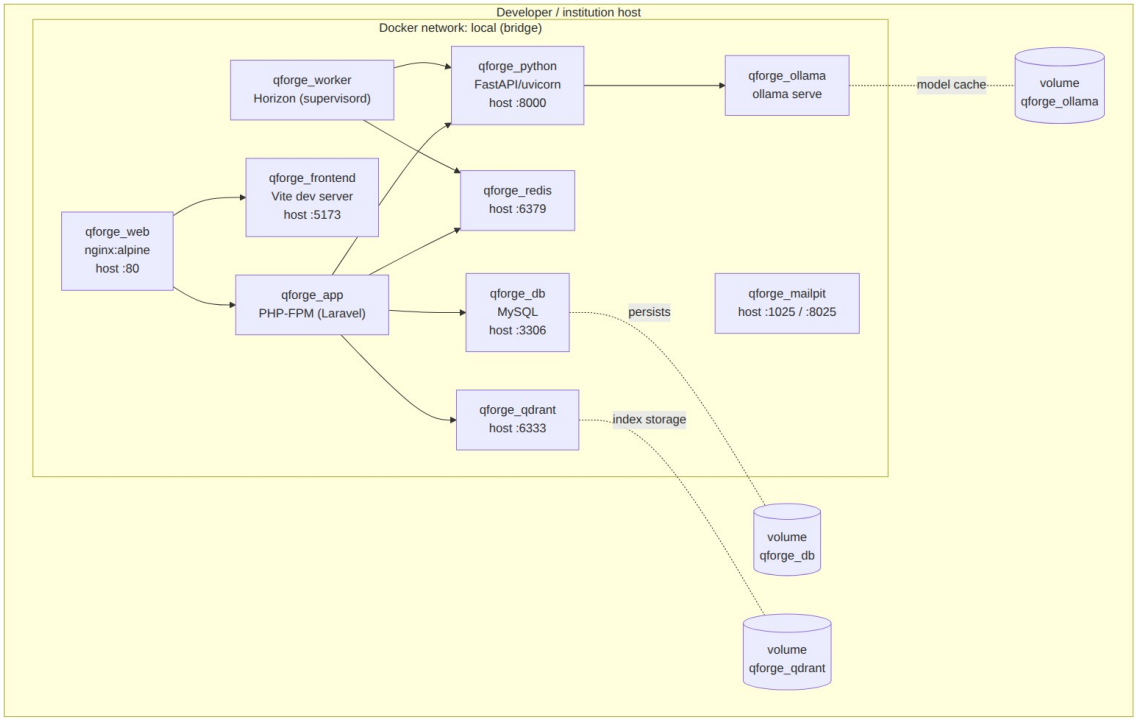

Figure 4.7: Deployment diagram — the Docker Compose container topology, ports, and named volumes.

Database Design

Beneath the object-oriented design sits a relational persistence schema. In keeping with the object-oriented commitment of this report, the entity–relationship schema is presented here only as the *persistence view* of the design — the shape the domain objects take when stored — and never as a primary analysis model. The schema, shown in Figure 4.8, mirrors the domain classes of the analysis chapter one-to-one: `users`, `subjects`, `units`, `questions`, `blueprints`, `papers`, `paper_questions`, and `document_uploads`, together with the two M6 additions — the `question_unit`

junction that realises multi-unit tagging and the `content_chunks` retrieval corpus whose rows have a vector twin in Qdrant.

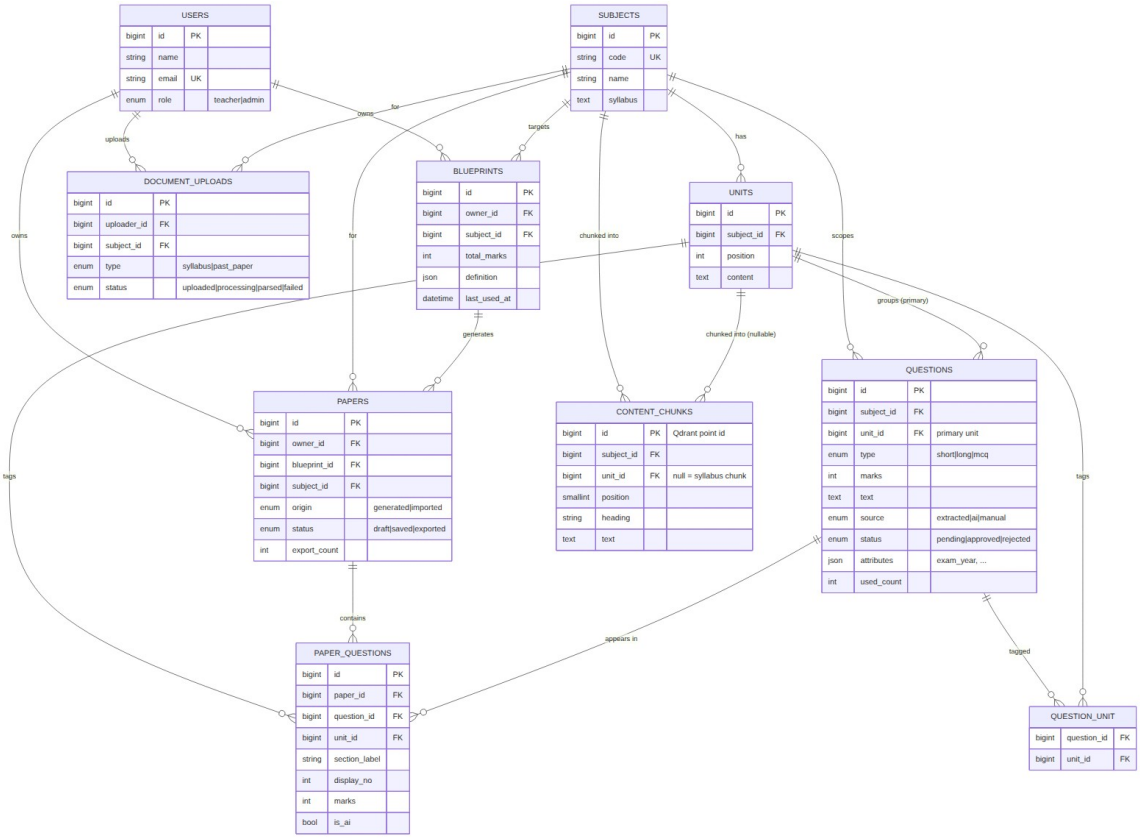

Figure 4.8: Relational schema — the persistence view of the object-oriented design.

Normalization. The schema is in third normal form. Every table has a surrogate `bigint` primary key; all non-key attributes depend on the whole key and on nothing but the key. The one relationship that is genuinely many-to-many — a question spanning several units — is resolved through the `question_unit` junction table with a composite uniqueness constraint on `(question_id, unit_id)`, rather than by repeating unit references on the question row. The `questions.unit_id` column is retained as the question's *primary* unit for display and default tagging, and is not a normalization violation because the many-to-many membership lives entirely in the junction. Two deliberate, well-understood denormalizations remain for performance and audit. First, `questions.used_count` is a maintained counter of how many saved papers have used a question; it duplicates information derivable by counting `paper_questions` rows, but it exists so the least-recently-used ordering can be expressed as a cheap column sort rather than an aggregate on every candidate query. Second, each `paper_questions` row snapshots the `section_label`, `display_no`, `marks`, `unit_id`, and `is_ai` of the

question *at the moment of generation*; this repeats data that could be looked up live, but it is intentional, because a saved paper is a historical record that must not silently change if the underlying question is later edited or re-tagged. The `blueprints.definition` and `questions.attributes` columns hold semi-structured JSON — the blueprint specification and provenance metadata such as `exam_year` — where a rigid columnar decomposition would add no query value and would couple the schema to a still-evolving specification. The `content_chunks` table is pure derived data: it is wholesale-rebuilt from `units.content` and `subjects.syllabus` on demand, nothing foreign-keys into it, and its vector counterparts in Qdrant are rebuildable from it, so it sits outside the normalized core as a disposable retrieval index.

4.2 Algorithm Details

The paper-generation algorithm is the academic centrepiece of QForge and the substance of this chapter. It is a **deterministic, seed-parameterised, hybrid greedy plus backtracking** engine, implemented in plain PHP with no external constraint solver. Given a blueprint and the approved question bank it produces one of exactly two outcomes: a *valid paper* — every slot filled, every constraint satisfied — or a *best-effort partial paper accompanied by a precise account of what the bank lacks*. It never returns an invalid paper silently and never fabricates a question to fill a gap. The design stance that governs everything below is that **the algorithm owns the final paper**: selection is rule-based by construction — coverage is a hard rank, repetition a hard exclusion, unit overload a hard cap — rather than a tuned additive score whose weights might quietly trade one requirement away. The one place a learned signal reaches into the engine, the semantic-duplicate guard, does so only as a *soft, fail-open preference* that can break a tie toward variety but can never cause a shortfall.

The engine borrows two classical techniques and combines them: a greedy selection heuristic [7] for the fast common case, and a bounded backtracking search [6] to repair the cases the greedy pass cannot solve on its own. What follows treats the five phases in order, then determinism, the similarity guard, and complexity.

Vocabulary

A **blueprint** is a teacher-owned template with four parts: `sections` (the authoritative paper structure), `unitRules` (the allowed units), `unitAllocations` (per-unit

maximum caps), and `exclusionRules` (repetition windows). A **slot** is one position in the paper to be filled by exactly one question, carrying its section label, normalised type, marks, and display number; sections are flattened into an ordered list of slots. A **candidate** is an approved question matching a slot's subject, type, and marks whose tagged units overlap the allowed set. **Coverage** is the hard rule that every allowed unit appears at least once in the finished paper. The output is a **GenerationResult**: either a valid paper with an all-pass constraint checklist, or a partial paper with a list of **missing slots** naming the shortfall.

Phase 0 — Compile

`BlueprintCompiler::compile` deserialises the stored blueprint into a `CompiledBlueprint` value object holding everything the later phases need. Each `section` of count *n* is expanded into *n* single-question slots in section order, with the display type normalised to a canonical question type (`"Short Answer"` to `short`, `"Long Answer"` to `long`, `"MCQ"` to `mcq`) and a running one-based display number. The truthy `unitRules` names are resolved to `allowedUnitIds` within the subject; an empty set means no unit restriction and, consequently, no coverage requirement. The `unitAllocations` counts are compiled into per-unit maximum caps — a unit's cap is the sum of its rows' counts; a unit with no rows is uncapped, and the marks column of those rows is display-only and never enforced. Finally the repetition windows — `lastNPapers` and `excludeExamYearsBack` — are captured. The compiled blueprint also exposes the structural-feasibility predicates used in Phase 3.

```
function compile(blueprint):
    slots ← []
    displayNo ← 1
    for each section in blueprint.definition.sections:
        type ← normaliseType(section.type)
        for i in 1..section.count:
            slots.append(Slot(index, section.name, type,
section.marksEach, displayNo))
            displayNo ← displayNo + 1
    allowedUnitIds ← resolveUnits(blueprint.definition.unitRules)
//  empty  ⇒  no  restriction
    unitCaps ←
```

```
compileCaps(blueprint.definition.unitAllocations, allowedUnitIds)
                lastNPapers                                          ←
blueprint.definition.exclusionRules.lastNPapers       ??       0
                                        excludeExamYearsBack         ←
blueprint.definition.exclusionRules.excludeExamYearsBack   ??   0
        return CompiledBlueprint(subjectId, totalMarks, slots,
allowedUnitIds,
                                unitNames, unitCaps, lastNPapers,
excludeExamYearsBack)
```

Phase 1 — Greedy fill

`PaperGenerator::greedyFill` walks the slots in order and fills each with the best available candidate. For a slot, `CandidateFilter::for` builds the pool: approved questions of the subject whose type and marks match the slot, at least one of whose tagged units is allowed (the *any-overlap* rule, so a genuinely multi-topic question stays eligible), excluding every question already placed on this paper, and — through an injected exclusion closure — excluding questions caught by the cross-paper repetition windows. The generator composes two such windows: the **last-N papers** rule drops questions that appeared on the most recent *N* papers or imported past exams for the subject, and the **last-N exam years** rule drops questions whose recorded provenance year falls in the years immediately before the current one. Both are built as excluded-*id* sets so that questions with no recorded year are never dropped by accident. The pool is returned ordered least-recently-used first (`used_count` ascending, then `id`).

Three refinements then act on that pool before a pick is made. First, if the blueprint imposes per-unit caps, any candidate whose selection would push one of its tagged capped units past its maximum is rejected — a multi-unit question counts against every capped unit it is tagged with. Second, when a similarity guard is active and at least one question is already placed, the engine *prefers* candidates that are not near-duplicates of the placed questions, but only if discarding the duplicates leaves the pool non-empty; otherwise the full pool is kept. This is the soft duplicate guard, and its pool-non-emptying rule is what guarantees it can never cause a shortfall. Third, `GreedySelector::pick` chooses from the surviving pool by a total order: candidates that cover a still-uncovered allowed unit rank first, then lowest `used_count`, then the seeded tie-break. The chosen

question’s id is added to the in-paper exclusion set — which guarantees no repetition — and its tagged units are merged into the running covered set and cap usage.

```
function greedyFill(compiled, exclusions, seed, guard):
    selections ← {} ; usedIds ← [] ; covered ← {} ; unitUse ← {}
     for each slot in compiled.slots:                // in order
            pool ← CandidateFilter.for(slot, compiled, usedIds,
exclusions)
                                if compiled.unitCaps ≠ ∅:
            pool ← pool without candidates that would bust a cap
given                                                   unitUse
                                                guard.prime(pool)
                                      if selections ≠ ∅:
                               fresh ← pool without candidates
guard.tooSimilar(candidate,                            selections)
             if fresh not empty: pool ← fresh           // soft:
never                empty                the                pool
                uncovered ← compiled.allowedUnitIds − covered
          pick ← GreedySelector.pick(pool, uncovered, seed)   //
coverage,         then         LRU,         then         tie-break
                                        if pick ≠ null:
                                  selections[slot.index] ← pick
                                           usedIds.append(pick.id)
                             for unitId in pick.taggedUnitIds():
                                               covered.add(unitId)
                  if unitId ∈ compiled.unitCaps: unitUse[unitId] ←
unitUse[unitId]                            +                    1
    return selections
```

The greedy pass is coverage-aware but *myopic*: ranking uncovered units first makes it cover all units on the first pass for most feasible blueprints, but it cannot see how an early pick constrains later slots — a cap it exhausts early, or a marks-shape conflict several slots ahead — so an invalid greedy paper is still possible. Repairing exactly those cases is the purpose of Phase 3.

Phase 2 — Validate

`ConstraintValidator::validate` scores the current selections into a list of pass/fail lines, one per constraint, in the shape the frontend renders directly. It checks, per section, that the filled count equals the required count; that the summed marks of filled slots equal the blueprint's total marks; when units are restricted, that the number of distinct allowed units used equals the number of allowed units (union semantics — a multi-unit question covers every allowed unit it carries); when caps exist, that no capped unit is tagged by more than its maximum; and that no question id repeats within the paper. If every slot is filled *and* every constraint passes, the run succeeds and returns immediately.

```
function validate(compiled, selections) → ConstraintResult[]:
                                    results ← []
        for each sectionLabel: results.append(filledCount == requiredCount)
              results.append( Σ marks(filled slots) == compiled.totalMarks )
                    if compiled.allowedUnitIds ≠ ∅:
        coveredUnits ← distinct( ⋃ selections.taggedUnitIds() ) ∩ allowedUnitIds
           results.append( |coveredUnits| == |allowedUnitIds| ) // coverage
                        if compiled.unitCaps ≠ ∅:
           results.append( every capped unit used ≤ its cap ) // maximums
            results.append( no repeated question id )
    return results
```

Phase 3 — Backtracking repair

When the greedy paper is invalid, `BacktrackingResolver::resolve` searches for a fully-valid assignment by bounded depth-first search [6]. Before searching, a **structural short-circuit** is applied: if the `CompiledBlueprint` is structurally infeasible — the coverage rule demands more units than the paper can ever reach (`|allowedUnitIds| > 2 × |slots|`, because an AI top-up question spans at most two units), or every allowed unit is capped and the caps sum below the slot count — the search is skipped

entirely, because no assignment can succeed, and control passes straight to Phase 4. This avoids burning the iteration budget on a provably impossible problem.

Otherwise the resolver builds the full candidate pool per slot (with the cross-paper exclusions still applied, but without the per-paper exclusions, since it enforces uniqueness itself) and runs a DFS over the slots, treating the slot index as the search depth. At each node it orders the remaining candidates by the same key the greedy selector uses — uncovered-unit-first, then `used_count`, then the seeded tie-break — which actively steers the search toward coverage; it skips any candidate whose id is already on the current path, and prunes any that would exceed a per-unit cap on this path. On reaching the last slot it accepts the assignment only if coverage holds; otherwise it backtracks and tries the next candidate. The search is bounded by `MAX_ITERATIONS = 50000`, so it always terminates; the coverage-first ordering reaches a covering assignment quickly in practice, so the bound is rarely approached. The similarity guard is deliberately *not* consulted here — similarity is a soft preference, not a constraint, and enforcing it in the repair pass could turn a satisfiable paper infeasible, so the repair pass optimises for a valid assignment above all. A returned assignment is re-validated and the run succeeds.

```
function resolve(compiled, candidatesBySlot, seed):
                                    iterations ← 0
    return search(depth = 0, assigned = {}, usedIds = [], unitUse
=                                                                {})

function search(depth, assigned, usedIds, unitUse):
          if ++iterations > MAX_ITERATIONS: return null
                    if depth == |compiled.slots|:
        return coversAllUnits(assigned) ? assigned : null      //
accept              only              if              covered
                         slot ← compiled.slots[depth]
    uncovered ← compiled.allowedUnitIds − coveredUnits(assigned)
                  ordered ← candidatesBySlot[slot.index]
                        reject candidates whose id ∈ usedIds
                 reject candidates that would bust a cap given
unitUse
                          sort by (uncoveredRank, used_count,
TieBreaker.key(seed,                                       id))
```

```
                    for each question in ordered:
                                assigned[slot.index] ← question
                result ← search(depth + 1, assigned, usedIds +
question.id,          unitUse          +          question.units)
                      if result ≠ null: return result
        remove assigned[slot.index]                              //
backtrack
    return null
```

Phase 4 — Explain the shortfall

If backtracking also fails (or was short-circuited), the blueprint is infeasible against the current bank, and `PaperGenerator::computeMissingSlots` returns the best-effort partial paper together with a precise, actionable account of what is missing, computed in two tiers. The **supply deficit** tier is primary: slots are grouped by `(section, type, marks)`, and for each group the deficit is the required count minus the number of distinct approved questions available; every group with a positive deficit becomes a `MissingSlot` naming the type, marks, and shortfall count, targeted at the scarcest allowed unit (the two scarcest when the coverage rule demands more units than there are slots, so a single AI top-up question can span both and pull double duty). If supply is adequate everywhere but the paper still cannot be covered, the **coverage deficit** tier applies: when there are at least as many slots as units, each uncovered unit becomes a single-unit missing slot; when units outnumber slots, only unit-spanning questions can close the gap, so the scarcest units are paired — one missing slot per pair — with singles for any leftover uncovered units. Every `MissingSlot` carries its target unit ids ordered scarcest-first, which is exactly the input the AI top-up job consumes to generate questions on the right units.

```
function computeMissingSlots(compiled, partial, candidatesBySlot)
→                                                  MissingSlot[]:
             //   Tier   1   —   supply   deficit   (primary)
        groups  ←  slots  grouped  by  (section,  type,  marks)
                                    missing        ←        []
                                for        each        group:
                           deficit   ←   group.required   −   |
candidatesBySlot[group.slot.index]|
```

```
                                    if deficit > 0:
                span ← (|allowedUnitIds| > |slots|) ? 2 : 1
            targets ← scarcest `span` allowed units for this pool
            missing.append(MissingSlot(section, type, marks, need
=          deficit,          unitIds          =          targets))
                  if missing ≠ ∅: return missing
             // Tier 2 — coverage deficit (fallback)
        uncovered ← allowedUnitIds − ⋃ partial.taggedUnitIds()
                     if |allowedUnitIds| ≤ |slots|:
            chunks ← each uncovered unit as a single [unitId]
    else:                                                    //
pair               the               scarcest               units
                   pairCount ← |allowedUnitIds| − |slots|
          chunks     ← scarcest (2 × pairCount) units paired up,
plus              singles              for              leftovers
                    for each unitIds in chunks:
                   missing.append(MissingSlot(firstSlot.section,
firstSlot.type,                                  firstSlot.marks,
                                  need = 1, unitIds = unitIds))
    return missing
```

Phase 5 — Preview then Save

Generation is a two-step, preview-first flow, and the engine itself is pure — it never persists. The generate request draws a fresh random seed (or accepts a client seed to reproduce a paper), runs the engine, and returns the paper with a null id plus the seed and blueprint id; nothing is written, so repeatedly regenerating never clutters history or skews usage statistics. Infeasible responses have the same unpersisted shape, carrying the partial paper and the missing slots. Saving sends back the blueprint id and the seed; because the engine is deterministic given its seed, the save path *re-generates* rather than trusting a client-supplied selection, and within a transaction inserts the `papers` row and one `paper_questions` row per selection, increments each picked question's `used_count`, and stamps the blueprint's `last_used_at`. If the bank has changed so the seed no longer yields a full paper, the save is rejected rather than persisting something the teacher never previewed. The lifecycle is therefore preview (unsaved) to saved to exported, with no auto-created draft.

Determinism given the seed

Candidate ordering is a *total* order — `(uncoveredRank, used_count ascending, TieBreaker::key(seed, id))` — whose final component is unique per question id, so every tie chain terminates deterministically. The output is thus a pure function of the bank, the blueprint, and the seed: the same bank plus the same blueprint plus the *same seed* yields the same paper on every run, and a null seed reproduces the original identifier-ordered paper exactly, which is the property the unit tests pin. The seed is the *only* source of variation, and `TieBreaker::key` confines it to the last tie-break: for a null seed it returns the id itself, and for any integer seed it returns `crc32("seed:id")`, a stable, well-spread reordering of *only* those candidates the rank already treats as equal. Coverage, the least-recently-used rotation, the caps, and every constraint are never overridden by the seed. This is what lets a regenerate action and two teachers working simultaneously each obtain a different but equally valid paper, while any specific paper remains exactly reproducible from its recorded seed — the property that makes an examination paper auditable and defensible.

The soft semantic-duplicate guard

Identifier uniqueness prevents the *same row* from appearing twice on a paper, but it cannot detect two *different* rows that mean the same thing — “Define a binary tree” and “What is a binary tree?”. The `SimilarityGuard` closes that gap using the embedding vectors already maintained for the question bank in Qdrant. It is an interface with two implementations. `NullSimilarityGuard` is the default no-op, which keeps the engine a pure function of the bank for unit tests and the feasibility probe. `VectorSimilarityGuard` loads the candidate questions’ vectors on demand, caches them per run, and reports a candidate as too similar when its cosine similarity to any already-placed question reaches the configured `RAG_DUPLICATE_THRESHOLD` (0.90 by default). Two properties make the guard safe. It is applied in the greedy pass *only* and as a *pool-non-emptying preference*, so it can prefer variety but can never empty a slot’s pool or turn a satisfiable paper infeasible; and it *fails open* — if RAG is disabled, a vector is missing, or Qdrant is unreachable, the guard reports “not similar” and generation proceeds on identifier-uniqueness alone. It is the single point at which a learned, semantic signal reaches into selection, and only ever as far as breaking a tie toward a more varied paper — the deterministic rules remain fully in control.

Complexity

Let S be the number of slots and C the maximum number of candidates per slot. The greedy fill plus validation is $O(S \cdot C)$: one filtered query and a linear selection per slot. The backtracking repair is worst-case exponential in the number of slots, as any depth-first constraint search can be [6], but it is bounded by `MAX_ITERATIONS = 50000` so it always terminates, and the uncovered-unit-first ordering drives it to a covering assignment quickly in practice, so in the common case it runs far below that ceiling and is frequently never entered at all, because the coverage-aware greedy pass already produces a valid paper. The similarity guard adds, per run, at most one batched vector retrieval and a bounded number of cosine comparisons against the small set of already-placed questions, which does not change the asymptotic cost of generation.

Chapter 5: Implementation and Testing

This chapter records how the design of Chapter 4 was realised as a working system and how that system was verified. It first names the tools that were used and the role each played in QForge, then walks the principal modules — foregrounding the constraint-based generation engine that is the project's academic centerpiece — and finally presents the test suite as a table of concrete cases, mapped to the algorithm's phases, and evaluates the outcome against the objectives set out in Section 1.3.

5.1 Implementation

5.1.1 Tools Used

The tools below were not chosen in the abstract; each was selected for a specific responsibility in QForge's service-oriented architecture, in which **Laravel is the orchestrator, Python a stateless processor, and Vue a decoupled presentation layer**. They are described here in terms of that role.

Laravel 12 (PHP 8.2). Laravel is the source of truth. It owns the MySQL schema, the question bank, blueprints, papers, and — critically — the deterministic generation algorithm, which lives in plain PHP service classes under `app/Services/PaperGeneration/` rather than in any framework machinery. Laravel also exposes the entire public API, authenticates it with **Laravel Sanctum** token guards, and gates every route by role. It is the only service that talks to the database and the only one that initiates calls to Python.

Vue 3 with Pinia and Tailwind CSS 4. The single-page frontend is presentation only and speaks exclusively to the Laravel API over `axios`; it never contacts the Python service directly. **Pinia** holds the client-side state (the authenticated session, the working blueprint, the current paper preview), **Vue Router** drives the admin and teacher screens, and **Tailwind CSS** supplies the utility-first styling for the bank, blueprint editor, generate/preview, review-queue, and export views.

FastAPI (Python). The Python service is a self-contained document- and AI-processing worker with no business logic and no access to the main database. It exposes typed endpoints — `/extract`, `/generate-questions`, and `/embed` — whose request and response shapes are declared as **Pydantic** models and returned inside a uniform

`Envelope`, so that Laravel always receives structured, validated JSON rather than free text.

MySQL. The relational store behind Laravel holds the domain tables introduced across the milestones (`subjects`, `units`, `questions`, `blueprints`, `papers`, `paper_questions`, `document_uploads`, `content_chunks`, and the `question_unit` pivot). Schema evolution is managed entirely through Laravel migrations, keeping the persistence model versioned alongside the code.

Redis with Laravel Horizon. Heavy work is never run inside a web request. PDF extraction and AI bank-expansion are dispatched as queued jobs onto **Redis**, and **Horizon** supervises the queue workers and provides visibility into job throughput and failures. This is what lets an upload return immediately while OCR and parsing proceed in the background, with the frontend polling for progress.

Qdrant. The vector database stores 768-dimensional embeddings of approved questions and of syllabus content chunks. QForge queries it for four purposes: dropping semantic near-duplicates during AI expansion, flagging look-alike candidates in the review queue, retrieving grounding context for generation, and — through the within-paper similarity guard — steering the greedy pass away from placing two near-identical questions in the same paper.

Ollama. All AI inference is self-hosted through Ollama, which removes any per-call cost and keeps question content on-premises. Two models are used: `qwen2.5:3b-instruct` for candidate-question generation and `nomic-embed-text` for embeddings. This local-model choice is what makes the AI top-up economically viable for a single institution and is reflected in the feasibility analysis of Section 3.1.2.

Docker Compose. Every service — Laravel, the Vue build, FastAPI, MySQL, Redis, Qdrant, and Ollama — runs as a container on a shared Compose network, addressed by service name (for example, Laravel reaches Python through `config('services.python.base_url')`, never a hardcoded URL). This gives the whole system a single reproducible bring-up and underpins the portability claim among the non-functional requirements.

The following libraries carry specific processing responsibilities:

- **barryvdh/laravel-dompdf** renders a saved paper's view-model to a print-ready PDF.
- **phpoffice/phpword** renders the same view-model to an editable DOCX, so that a paper-setter can make final manual edits after export.
- **pdfplumber** and **pypdf** perform digital text and table extraction from uploaded PDFs — the fast path taken when a document already carries a real text layer.
- **pytesseract** (with **Pillow**) provides the per-page OCR fallback that recovers text from scanned, image-only past papers when the digital path yields nothing.

5.1.2 Implementation Details of Modules

This section describes the real classes and methods that make up QForge. Representative source listings for the major components are reproduced in Appendix {{APPENDIX_SOURCE_SNIPPETS}}; the descriptions here are deliberately at the level of responsibilities and control flow rather than full code.

The generation engine

The generation engine is a pipeline of small, single-responsibility classes under `app/Services/PaperGeneration/`, coordinated by `PaperGenerator`. Its public entry point, `PaperGenerator::generate(Blueprint $blueprint, ?int $seed, ?SimilarityGuard $guard)`, returns a `GenerationResult` value object and executes the five phases established in Section 4.2.

Compilation. `BlueprintCompiler::compile()` turns the stored blueprint JSON into a `CompiledBlueprint` — an ordered list of `Slot` objects (each with a question type and mark value), the set of allowed unit identifiers resolved from unit *names*, and the per-unit maximum caps. It also records whether the blueprint is *structurally* infeasible (coverage demanding more units than the slots can carry, or capped units whose caps sum to fewer than the slots), a determination the generator later uses to avoid a futile search.

Candidate filtering. `CandidateFilter::for()` builds the eligible question pool for each slot, applying the composed exclusion closures — the last-*N*-papers rule (`lastNExclusion`) and the recent exam-year rule (`examYearExclusion`), which `PaperGenerator::composeExclusions()` merges into a single closure applied to *both* the greedy and backtracking pools so that repair can never reintroduce an excluded question.

Greedy fill. `GreedySelector::pick()` selects one question per slot, ranking candidates by the deterministic tuple *(uncovered-unit-first, `used_count`, tie-break key)* so the pass both spreads coverage across units and behaves as a least-recently-used chooser. Where a real similarity guard is supplied, the greedy pass *prefers* candidates that are not near-duplicates of questions already placed in the paper — a soft preference that never empties a pool.

Validation. `ConstraintValidator::validate()` checks a completed assignment against every hard rule — slot counts, total marks, unit coverage, unit maximums, and within-paper non-repetition — and returns a list of `ConstraintResult` lines, each a named pass/fail. `allPass()` collapses them to a single verdict. This is what makes a paper *explainable*: the constraint checklist is surfaced to the user rather than hidden inside the engine.

Backtracking repair. When the greedy assignment is incomplete or fails validation and the blueprint is not structurally infeasible, `BacktrackingResolver::resolve()` runs a depth-first search over the per-slot candidate pools, pruning branches that violate a cap and confirming full unit coverage before accepting an assignment. It recovers valid papers that the naive greedy pass alone cannot find. Similarity is deliberately *not* enforced here, so that a soft preference can never turn a satisfiable paper infeasible.

Shortfall explanation. If no valid assignment exists, `PaperGenerator::computeMissingSlots()` returns a precise account of what is missing as an ordered list of `MissingSlot` objects — pairing the scarcest units when the blueprint asks for more units than it has slots — rather than a bare failure.

Determinism and the soft guard. `Support/TieBreaker::key(?int $seed, int $id)` resolves the final tie between otherwise-equal candidates: a null seed preserves the legacy identifier order (so existing tests are unaffected), while any given seed produces a reproducible per-seed shuffle. This is what lets two paper-setters, or a “regenerate” click, obtain different but each fully reproducible papers from one blueprint. The optional within-paper duplicate guard is defined by the `Contracts/SimilarityGuard` interface with two implementations: `NullSimilarityGuard`, the default no-op, and `VectorSimilarityGuard`, which primes itself with the vectors of the chosen questions (`prime()`) and answers `tooSimilar()` by cosine comparison against Qdrant. The guard fails open — if RAG is unavailable it simply behaves as the null guard.

API controllers

`PaperController` exposes the generation lifecycle. Its `generate()` action draws a fresh `random_int` seed per request (or accepts a client-supplied one), runs the engine, and returns an *unsaved* preview (`id: null`) together with the seed and blueprint id — it persists nothing. `store()` requires a seed and re-generates deterministically from it before persisting the paper as `status = saved`, bumping `used_count` and `last_used_at` only at that point; because the engine is a pure function of (blueprint, seed, bank), the saved paper matches the preview. The controller also serves the owner-scoped `index`, `show`, `update`, `destroy`, `analytics`, and `export` actions, each gated by role and ownership. The resource controllers for subjects, units, questions, and blueprints follow the same Sanctum-authenticated, role-gated pattern.

Python endpoints

`POST /extract` accepts a path (validated to lie inside the shared volume, so traversal is refused), extracts a digital text and table layer with pdfplumber, falls back to per-page Tesseract OCR when a page carries no usable text, runs the heuristic parser to segment the text into candidate questions with detected marks, types, and unit hints, and returns them — with any parsed syllabus courses — inside a typed `Envelope`. `POST /generate-questions` validates a bounded request (rejecting a count out of range or more than two units), grounds the prompt, and delegates to an env-selected `LLMProvider` — `OllamaProvider` in production or `StubProvider` under test — returning structured questions, with any malformed model item isolated into an `errors` list rather than allowed to corrupt the `data`.

Background jobs

Two queued jobs keep heavy work off the request path. `ProcessDocumentUpload::handle()` calls the Python `/extract` endpoint for an uploaded document, records progress on the `document_uploads` row, and imports the returned candidates into the review queue via `CandidateImporter` — never directly into the approved bank. `ExpandQuestionBank::handle()` is dispatched as a batch from `POST /blueprints/{id}/expand-bank`; it re-derives the missing slots server-side, builds grounding context with `GroundingBuilder` (unit content, syllabus, and up to a few approved exemplars), asks Python for slightly more questions than needed, drops

semantic near-duplicates through `DuplicateDetector`, stamps each survivor's type and marks from the slot, resolves its unit tags, and stores it as `source = ai, status = approved` — closing a feasibility gap while keeping the AI strictly supportive of the algorithm.

Export

`Support/PaperViewModel` wraps a saved `Paper` and exposes a rendering-agnostic view of it — `header()`, `sections()`, `toArray()`, and a `filename()` — so that both export formats draw from one source. `PaperController::export()` selects the renderer by the `format` query parameter: dompdf for PDF and PhpWord for DOCX. An unsupported format is rejected rather than silently defaulted.

Supporting RAG infrastructure

The retrieval components that surround the engine — `Services/Rag/QdrantClient`, the queued `SyncQuestionEmbedding`/`SyncSubjectChunks` synchronisers driven by model observers, the `ContentChunker`/`ContentIndexer` grounding index, and the `SimilarQuestionFinder` review-queue flagger — are named here as the supporting cast that keeps the vector index consistent with the approved bank and the syllabus corpus. All of them fail open behind a `RAG_ENABLED` switch, so a degraded RAG layer never blocks core generation.

5.2 Testing

Testing followed the milestone-first build: each milestone shipped with its own green suite, and the whole was exercised end to end at every stage. The completed system carries **225 Laravel tests and 153 pytest tests, all green** (Post-M6, as recorded in the milestone log). The two subsections below present the cases as tables, mapping the unit tests to the algorithm's phases and the system tests to the API and extraction pipeline; each row names the actual test method that backs it, and every case listed is passing. Both deliberate positive and deliberate negative cases are included — an infeasible blueprint, an over-capped blueprint, an invalid upload, an unsupported export format, and a path-traversal attempt are all asserted to fail in the intended way.

5.2.1 Unit Testing

The unit suite under `tests/Unit/PaperGeneration/` isolates the generation engine from HTTP and the database factories, driving it directly so that each algorithm phase is verified on its own. Table 5.1 maps these cases to the phases of Section 4.2.

Table 5.1: Unit test cases for the generation engine

Test ID	Scenario	Input	Expected	Actual	Result
UT-01	Feasible blueprint yields an all-green paper (`PaperGeneratorTest::test_generates_a_valid_paper_with_all_constraints_passing_for_a_feasible_blueprint`)	Feasible blueprint; bank large enough to fill every slot	Complete paper; every `constraint_results` line passes; `missing_slots` empty	As expected	Pass
UT-02	Unit coverage enforced (`PaperGeneratorTest::test_enforces_unit_coverage_every_allowed_`	Blueprint whose allowed units must all appear	Each allowed unit is represented in the selected questions	As expected	Pass

Test ID	Scenario	Input	Expected	Actual	Result
	`unit_appe ars`)				
UT-03	No question repeats within one paper (`PaperGen eratorTes t::test_n ever_repe ats_a_que stion_wit hin_a_sin gle_paper`)	Blueprint with more slots than distinct near-duplicates	All selected question ids are distinct	As expected	Pass
UT-04 *(negative)*	Thin bank reports a precise shortfall (`PaperGen eratorTes t::test_r eports_a_ precise_s hortfall_ when_the_ bank_is_t oo_thin`)	Blueprint requiring more questions than the bank holds	`satisfiab le = false` with populated, named `missing_s lots`	As expected	Pass
UT-05	Backtrackin g recovers a paper greedy misses	Bank arranged so a greedy pick strands	A fully valid assignment is found via repair	As expected	Pass

Test ID	Scenario	Input	Expected	Actual	Result
	(`PaperGeneratorTest::test_backtracking_recovers_a_valid_paper_the_naive_greedy_pass_fails_to_find`)	a later slot			
UT-06	Per-unit maximum cap honoured (`PaperGeneratorTest::test_unit_cap_limits_how_many_questions_a_unit_contributes`)	Blueprint capping one unit's contribution	No unit exceeds its cap; "Unit maximums" line passes	As expected	Pass
UT-07 *(negative)*	Caps summing below slots are infeasible (`PaperGeneratorTest::test_caps_summi`	Every unit capped; Σcaps < slots	Flagged structurally infeasible; no backtrack attempted	As expected	Pass

Test ID	Scenario	Input	Expected	Actual	Result
	`ng_below_the_slot_count_are_structurally_infeasible`)				
UT-08	Cross-paper exclusion of recent papers (`PaperGeneratorTest::test_excludes_questions_used_in_the_last_n_papers`)	Blueprint with a last-*N*-papers window and prior papers	Questions from the last *N* papers never reappear	As expected	Pass
UT-09	Recent exam-year exclusion (`PaperGeneratorTest::test_excludes_questions_from_a_recent_exam_year`)	Rule excluding questions from the last *N* exam years	Year-tagged questions in the window are dropped; year-less ones stay eligible	As expected	Pass
UT-10	Seeded variation is reproducible (`PaperGen`	One blueprint generated under equal	Same seed → identical paper; different	As expected	Pass

Test ID	Scenario	Input	Expected	Actual	Result
	`eratorTest::test_same_seed_is_reproducible_and_different_seeds_vary_the_paper`; `TieBreakerTest::test_a_given_seed_is_stable_across_calls`, `::test_different_seeds_reorder_the_same_ids`)	and differing seeds	seed → a different valid paper		
UT-11	Soft within-paper dedup never causes a shortfall (`PaperGeneratorTest::test_greedy_pass_avoids_a_near_duplicate_when_an_alternative`	Bank with a near-duplicate plus a distinct alternative	The distinct alternative is preferred; paper still fills	As expected	Pass

Test ID	Scenario	Input	Expected	Actual	Result
	`_exists`, `::test_duplicate_avoidance_never_causes_a_shortfall`)				
UT-12	Similarity guard fails open and flags by cosine (`VectorSimilarityGuardTest::test_fails_open_when_rag_is_disabled`, `::test_flags_near_duplicates_by_cosine_over_the_threshold`, `::test_distinct_directions_are_not_duplicates`)	Guard with RAG disabled, then with vectors above/below threshold	Disabled → no-op; above threshold → flagged; distinct → not flagged	As expected	Pass

5.2.2 System Testing

The system suite exercises the running services through their public surfaces: Laravel feature tests issue authenticated HTTP requests against the API (asserting behaviour, persistence, role gating, and export bytes), while the pytest suite drives the Python service against real digital and scanned PDF fixtures, including OCR. Table 5.2 lists representative cases; the `ST-` rows are Laravel feature tests and the `PY-` rows are pytest cases.

Table 5.2: System test cases (API feature tests and Python extraction)

Test ID	Scenario	Input	Expected	Actual	Result
ST-01	Generate returns a preview and persists nothing (`PaperGenerateTest::test_generate_returns_a_preview_and_persists_nothing`)	`POST /papers/generate` for a feasible blueprint	Paper with `id: null` + a `seed`; no `papers` row written	As expected	Pass
ST-02	Saving a preview persists it (`PaperGenerateTest::test_saving_a_preview_persists_it_and_bumps`	`POST /papers` with the preview's seed	Paper saved as `status=saved`; `used_count` bumped	As expected	Pass

Test ID	Scenario	Input	Expected	Actual	Result
	`_used_count`)				
ST-03 *(negative)*	Infeasible blueprint returns missing slots, persists nothing (`PaperGenerateTest::test_returns_missing_slots_and_persists_nothing_for_infeasible_blueprint`)	`POST /papers/generate` for an unsatisfiable blueprint	`missing_slots` returned; no paper persisted	As expected	Pass
ST-04 *(negative)*	Generation is role- and owner-gated (`PaperGenerateTest::test_admin_cannot_generate_papers`, `::test_cannot_generate_from_another_`	Admin token; another teacher's blueprint	Requests forbidden	As expected	Pass

Test ID	Scenario	Input	Expected	Actual	Result
	`teachers_blueprint`)				
ST-05	Repeated generation excludes recent questions (`PaperGenerateTest::test_second_generation_excludes_questions_from_the_last_n_papers`)	Two successive generate→save cycles	Second paper omits the first's questions	As expected	Pass
ST-06	PDF export succeeds and marks the paper (`PaperExportTest::test_export_returns_a_valid_pdf_and_marks_the_paper_exported`)	`GET /papers/{id}/export?format=pdf` on a saved paper	Valid PDF bytes; paper flagged exported	As expected	Pass
ST-07	DOCX export	`...export?`	Valid DOCX;	As expected	Pass

Test ID	Scenario	Input	Expected	Actual	Result
	succeeds (`PaperExportTest::test_export_returns_a_valid_docx_and_increments_export_count`)	`format=docx`	export count incremented		
ST-08 *(negative)*	Unsupported export format is rejected (`PaperExportTest::test_export_rejects_an_unsupported_format`)	`...export?format=xlsx`	Request rejected, no file produced	As expected	Pass
ST-09	Upload stores the file and queues extraction (`DocumentUploadApiTest::test_upload_stores_the_file_and_dispatc`	`POST /uploads` with a PDF	File saved to shared volume; `ProcessDocumentUpload` dispatched	As expected	Pass

Test ID	Scenario	Input	Expected	Actual	Result
	`hes_the_extraction_job`)				
ST-10 *(negative)*	Non-PDF upload refused (`DocumentUploadApiTest::test_only_pdfs_are_accepted`)	Upload of a non-PDF file	Validation error; nothing stored	As expected	Pass
ST-11 *(negative)*	Teachers cannot upload (`DocumentUploadApiTest::test_teachers_cannot_upload`)	Upload with a teacher token	Forbidden	As expected	Pass
ST-12	AI expansion dispatches a batch for an infeasible blueprint (`BankExpansionTest::test_expand_dispatches_a_batch_for`	`POST /blueprints/{id}/expand-bank` on an infeasible blueprint	Batch dispatched; `{ jobId }` returned	As expected	Pass

Test ID	Scenario	Input	Expected	Actual	Result
	`_an_infeasible_blueprint`)				
ST-13 *(negative)*	Structurally infeasible blueprint refused for expansion (`BankExpansionTest::test_expand_refuses_a_structurally_infeasible_blueprint`)	Expand on a structurally impossible blueprint	Refused as not expandable	As expected	Pass
PY-01	Digital extraction without OCR (`test_extract.py::test_returns_candidates_without_ocr`)	PDF with a real text layer	Candidate questions returned via the fast path	As expected	Pass
PY-02	Marks, types, and units detected (`test_extract.py::`	Past-paper PDF with mark and unit annotations	Candidates carry parsed marks, types, unit hints	As expected	Pass

Test ID	Scenario	Input	Expected	Actual	Result
	`test_detects_marks_types_and_units`)				
PY-03	OCR fallback recovers scanned text (`test_extract.py::test_ocr_fallback_recovers_text_from_an_image_only_pdf`)	Image-only (scanned) PDF	Text recovered by Tesseract; candidates produced	As expected	Pass
PY-04 *(negative)*	Letterhead is not treated as a question (`test_extract.py::test_letterhead_is_not_a_candidate`)	PDF with header/letterhead text	Non-question text excluded from candidates	As expected	Pass
PY-05 *(negative)*	Path traversal refused (`test_extract.py::test_trav`	Path escaping the shared volume	Request refused	As expected	Pass

Test ID	Scenario	Input	Expected	Actual	Result
	`ersal_out_of_the_volume_is_refused`, `::test_path_outside_the_shared_volume_is_refused`)				
PY-06 *(negative)*	Non-PDF reports an error, does not raise (`test_extract.py::test_a_non_pdf_reports_an_error_rather_than_raising`)	A non-PDF file path	Structured error in the envelope; no crash	As expected	Pass
PY-07	Generation returns valid structured questions (`test_generate.py::test_returns_valid_structured_questions`)	Valid generate request (stub provider)	Well-formed questions in `data`	As expected	Pass

Test ID	Scenario	Input	Expected	Actual	Result
PY-08 *(negative)*	More than two units rejected (`test_generate.py::test_rejects_more_than_two_units`)	Request naming three units	Request rejected	As expected	Pass
PY-09 *(negative)*	Malformed model item isolated (`test_generate.py::test_malformed_item_lands_in_errors_not_data`)	Provider returns one malformed item	Malformed item lands in `errors`, not `data`	As expected	Pass

5.3 Result Analysis

The system meets each objective stated in Section 1.3. The results are discussed below against those objectives in turn; representative outputs are cross-referenced to the Appendix.

A constraint-based, blueprint-driven generator producing valid, balanced papers. The engine produces a complete paper in which every slot is filled and the per-section counts and total marks are satisfied, with a per-constraint checklist rendered green (UT-01, ST-01). Where no such paper exists, it returns a precise, named account of the shortfall rather than a partial or invalid paper (UT-04, ST-03). A representative all-green paper with its constraint checklist is reproduced in Appendix {{APPENDIX_PAPER_OUTPUT}} (Figure {{FIG_PAPER_OUTPUT}}).

Enforced unit coverage, per-unit caps, and cross-paper non-repetition. Coverage, unit maximums, and exclusion of recently used questions are enforced *as rules of generation* rather than checked afterwards: every allowed unit appears (UT-02), no unit exceeds its cap (UT-06), and successive papers from one blueprint draw on disjoint questions (UT-08, ST-05). The coverage-aware greedy pass with backtracking repair recovers valid papers a naive selection would miss (UT-05).

Reproducible and explainable generation. Generation is deterministic given a recorded seed: the same seed reproduces a paper exactly, while a different seed yields a different valid paper (UT-10), and the preview/save lifecycle re-generates from the stored seed so a saved paper matches its preview (ST-01, ST-02). Every run carries a per-constraint pass/fail result, and every failure is expressed as named missing slots — the property that makes any outcome auditable.

Ingesting past-paper and syllabus documents into an admin-reviewed bank. The extraction pipeline recovers questions from both digital and scanned PDFs — taking the OCR fallback when needed — and detects marks, types, and unit hints (PY-01, PY-02, PY-03), while rejecting non-documents and unsafe paths (PY-05, PY-06). Extracted items enter a review queue rather than the approved bank, and uploads are validated and role-gated (ST-09, ST-10, ST-11). A representative review queue with extracted candidates is shown in Appendix {{APPENDIX_REVIEW_QUEUE}} (Figure {{FIG_REVIEW_QUEUE}}).

Optional, supportive AI top-up and export. When a blueprint is otherwise infeasible, AI expansion generates grounded candidate questions, guarded against semantic duplicates, and closes the feasibility gap without ever overriding the algorithm (ST-12); structurally impossible blueprints are refused rather than papered over (ST-13). Finished papers export to both PDF and DOCX from a single view-model, with an unsupported format rejected (ST-06, ST-07, ST-08). Sample PDF and DOCX exports appear in Appendix {{APPENDIX_EXPORTS}} (Figure {{FIG_EXPORTS}}).

Taken together, the green unit and system suites — 225 Laravel and 153 pytest tests — confirm that the deterministic algorithm owns the final paper, that AI remains strictly supportive, and that each Section 1.3 objective was delivered and verified against the running system.

Chapter 6: Conclusion and Future Recommendations

6.1 Conclusion

This project set out to remove the manual burden of examination paper setting without removing the teacher's control over the result. Preparing a question paper by hand was identified as a constraint-laden task that becomes slow and error-prone precisely because structure, unit coverage, mark distribution, question-type mix, and non-repetition must all be satisfied at once and checked against one another — coupled together in one person's head, where a change to satisfy one requirement can silently break another. QForge was built to answer that problem by letting a teacher describe the *shape* of an examination once, as a reusable blueprint, and then assembling a valid, balanced paper from a bank of approved questions automatically, with a deterministic algorithm — not an AI model — in final control of every selection.

The system was built iteratively through six principal milestones, sequenced algorithm-first so that the highest-risk component, the constraint-based generation engine, was implemented and proven early against seeded data before the messier document-processing and AI pipelines were layered on top of a foundation already known to be correct. The generation engine was written from scratch as a set of framework-light, individually testable classes: a compiler that flattens a blueprint into ordered slots, a candidate filter, a coverage-aware greedy selector, a bounded backtracking resolver, and a constraint validator. This hybrid greedy-plus-backtracking design — a fast path with a guaranteed fallback — was the deliberate choice over pure-random selection, brute force, and an external constraint solver, because it is at once fast on the common case, complete when a valid paper exists within its iteration bound, and, above all, deterministic and explainable rather than a black box. Around that engine the project delivered the full path from a curated question bank to a finished paper: role-based catalog and bank management, a paper lifecycle with preview-then-save and PDF and DOCX export, an admin-reviewed extraction pipeline for past-paper and syllabus PDFs, an optional local-AI top-up for thin banks, and a retrieval-augmented layer that grounds that top-up and guards against semantic duplicates.

The project's contribution is a working demonstration that a **hybrid system** — deterministic, rule-based logic augmented by supportive AI — can produce examination

papers that are provably valid without surrendering authority to a language model. The centrepiece is the generation algorithm itself: a from-scratch, seed-parameterised constraint solver that treats paper setting as a constraint-satisfaction problem, enforces coverage and repetition as hard rules rather than after-the-fact checks, and reports for every paper exactly which constraints passed and, for every failure, exactly what the bank was missing. The AI is deliberately positioned as supportive, not authoritative: it only ever proposes candidate question text, which is then stamped with type, marks, and unit and subjected to the same constraints as any other question, so it can never cause an invalid paper.

Each of the objectives stated in Section 1.3 was met by the delivered system:

- **A constraint-based, blueprint-driven generator was built.** The engine compiles a reusable blueprint into slots and produces a paper in which every slot is filled and the required per-section counts and total marks are satisfied; when no such paper exists, it returns a precise account of the shortfall as named missing slots rather than failing silently. The correctness of both the feasible and infeasible paths is pinned by the unit-test suite.

- **Unit coverage, per-unit maximum caps, and cross-paper non-repetition are enforced as hard rules of generation.** Coverage is driven into the greedy pass and guaranteed by the backtracker; per-unit maximums bound unit dominance; and questions used in recent saved papers — and, optionally, in recent examination years — are excluded at candidate-selection time, so structure, balance, and freedom from recent repetition are guaranteed by construction rather than checked afterwards.

- **Generation was made reproducible and explainable.** A run is deterministic given a recorded seed, which the system draws, echoes back, and can accept again to reproduce a paper exactly; and every paper carries a per-constraint pass/fail checklist, so any result can be audited and any failure is stated as named missing slots.

- **Past-paper and syllabus documents can be ingested into an admin-reviewed bank.** The extraction pipeline reads digital text with an optical-character-recognition fallback and heuristic parsing, turning past papers into candidate

questions and syllabi into subjects and units; nothing becomes examinable until an admin approves it, so the bank grows from existing material without unverified questions entering it.

- **An optional AI top-up and export were delivered.** When a blueprint is otherwise infeasible, a local language model expands the bank with grounded candidate questions, guarded against semantic duplicates and kept strictly under the algorithm's control; and finished papers export to both PDF and DOCX. No objective was left unmet.

The major findings of the work follow directly from these outcomes. A hand-written constraint solver, kept deterministic and self-contained, was sufficient to generate valid, balanced papers for realistic blueprints without any external solver dependency. The greedy-plus-backtracking split proved to be the right shape: the coverage-aware greedy pass solves most papers outright in a single linear sweep, and the backtracker is exercised only as a correctness safety net. Making the engine pure with respect to writes — reading the bank but never persisting — kept it unit-testable in isolation and allowed later additions, including the entire AI and retrieval-augmented layer, to be built on top without regressing the core, a property confirmed by a test suite that grew to 225 Laravel tests and 153 Python tests, all green. Finally, the document-processing work showed that robust extraction from real, messy TU papers is achievable with disciplined heuristics and a mandatory human review step, at the cost of full automation — a trade the design accepts by choice. Taken together, the results confirm the project's central claim: a hybrid system in which the deterministic algorithm, not the AI, owns the final paper is both buildable and defensible.

6.2 Future Recommendations

The system as delivered is complete against its objectives, but several deliberate design choices and honest limitations mark out a roadmap for future work.

- **A smarter or exact solver behind the same interface.** The backtracking resolver is worst-case exponential and is therefore bounded by a fixed iteration cap, which guarantees termination but could, on a very large bank with tightly coupled coverage rules, return a best-effort partial result even where a solution exists deep in the search tree. Because the resolver sits behind a narrow interface, a stronger

variable-ordering heuristic or a proper constraint-programming solver could be substituted without disturbing the rest of the engine if scale ever demands it.

- **Optimal multi-unit coverage in the AI top-up.** When a blueprint mandates more units than it has slots, the top-up pairs the scarcest units greedily so that each requested question can cover two units at once. This pairing is a heuristic rather than a provably optimal set cover; it is self-correcting across expand-then-regenerate rounds, but a future version could compute a minimal covering assignment directly and could relax the two-units-per-question bound where genuinely integrative questions are appropriate.

- **Routing AI questions through human review.** For the demonstration loop, AI-generated questions are stored as approved so that a regenerate immediately succeeds. A production deployment should route them through the same admin review queue that extracted questions already pass through, so that a human confirms every AI-authored question before it can appear on an examination, closing the gap that AI question quality is currently grounded but not human-verified.

- **Model and provider flexibility.** The AI path depends on a single local model, though it already sits behind a provider abstraction with a stub implementation, so the model is swappable by configuration alone. Future work could support multiple providers and models, and could compare their output quality, while preserving the property that the entire AI path remains optional and the system functions fully without it.

- **Stronger and multilingual semantic duplicate detection.** The semantic-duplicate guard is threshold-based: it catches typical paraphrases but is not an absolute guarantee, and it fails open, so during an outage of the vector index or embedder a paraphrase could slip in until the next re-index. The embedding model is also fixed and tuned for an English corpus. Future work could add a more robust duplicate check that does not degrade silently, and could adopt a multilingual embedding model — a change the system is already prepared for, since every stored vector records the model that produced it.

- **Broader testing and more resilient extraction.** The algorithm core is thoroughly unit-tested, but broader end-to-end and Python-side coverage — especially against unusual and heavily degraded PDF layouts — is the honest gap and a natural next area to grow. The extraction parser is regex- and heuristic-based by design; extending it to more document templates, and optionally introducing AI-assisted extraction behind the existing mandatory review step, would widen the range of material the bank can absorb without weakening the human-in-the-loop guarantee.

- **Multi-institution and multi-format operation.** The present system assumes a single-institution deployment with TU-style, unit-and-marks-structured papers and an English-language corpus. Future work could generalise it to serve multiple institutions and to accommodate other examination formats, and could take the queue infrastructure — single-node here, which suits a college deployment — to a clustered, multi-worker configuration for larger scale, a change the chosen stack supports by configuration rather than redesign.

Each of these directions builds on, rather than revises, the architecture the project established: a deterministic algorithm at the centre, AI in a strictly supportive role, and clear service boundaries that keep each concern independently improvable.

References & Bibliography

Reconciled, final-order list. Every `[n]` used in the drafted chapters (Ch1–Ch6) is reconciled into the single ordered **References** list below, **renumbered in order of first appearance** across the concatenated report. The front matter, Ch1, Ch3, Ch5, and Ch6 contain no numeric citations; all in-text `[n]` markers occur in Ch2 (§2.1–§2.2) and Ch4 (§4.2, which reuses two Ch2 sources). A **renumbering map** (old draft `[n]` → new `[n]`) is given so the chapter files can be updated. The working bank with per-source confidence stays in `references.md`; this file is the shippable list.

Rule (GUIDE §5 rule 6, §6): every entry in the live References list is a real, checked source. Nothing is fabricated. Sources whose bibliographic details could not be verified are held in the **Candidates (quarantined)** section, *not* the live list, and their in-text slots remain `{{CITATION NEEDED}}`. Read-but-not-cited sources go in the **Bibliography**.

Renumbering map (old draft `[n]` → new `[n]`)

Apply these swaps to the chapter files. Only Ch2 and Ch4 are affected.

Old `[n]` (draft)	New `[n]` (final)	Source (short)	First appears
[7]	**[1]**	Cormen et al. — CLRS (greedy selection)	Ch2 §2.1.1
[6]	**[2]**	Russell & Norvig — AIMA (CSP / backtracking)	Ch2 §2.1.1
[4]	**[3]**	Smith — Tesseract OCR	Ch2 §2.1.3
[5]	**[4]**	Reimers & Gurevych — Sentence-BERT	Ch2 §2.1.4
[2]	**[5]**	Kurdi et al. —	Ch2 §2.1.5

Old `[n]` (draft)	New `[n]` (final)	Source (short)	First appears
		Automatic question generation survey	
[3]	**[6]**	Gierl & Haladyna — Automatic Item Generation	Ch2 §2.1.5
[1]	**[7]**	Liu, Wang & Zheng — Ant-colony test-paper generation	Ch2 §2.2

Per-file occurrences to update:

- **`02-background.md`** — replace, in this reading order: `[7]→[1]` (greedy, §2.1.1), `[6]→[2]` (backtracking, §2.1.1), `[4]→[3]` (OCR, §2.1.3), `[5]→[4]` (SBERT, §2.1.4), `[2]→[5]` and `[3]→[6]` (AQG/AIG, §2.1.5), `[1]→[7]` (random/metaheuristic baseline, §2.2). Later reuses in §2.2 — `[1]→[7]` (metaheuristic), `[6]→[2]` (CSP foundations), and `[2], [3]→[5], [6]` (generative family) — take the same new numbers.
- **`04-system-design.md`** — `[7]→[1]` (greedy, §4.2), `[6]→[2]` (backtracking, §4.2, three occurrences: engine intro, `BacktrackingResolver`, complexity).

References (IEEE — cited items only, first-appearance order)

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.

[2] S. J. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ, USA: Pearson, 2021.

[3] R. Smith, “An overview of the Tesseract OCR engine,” in *Proc. 9th Int. Conf. Document Analysis and Recognition (ICDAR)*, Curitiba, Brazil, Sep. 2007, pp. 629–633, doi: 10.1109/ICDAR.2007.4376991.

[4] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” in *Proc. 2019 Conf. Empirical Methods in Natural Language*

Processing and 9th Int. Joint Conf. Natural Language Processing (EMNLP-IJCNLP), Hong Kong, China, Nov. 2019, pp. 3982–3992.

[5] G. Kurdi, J. Leo, B. Parsia, U. Sattler, and S. Al-Emari, "A systematic review of automatic question generation for educational purposes," *International Journal of Artificial Intelligence in Education*, vol. 30, no. 1, pp. 121–204, Mar. 2020, doi: 10.1007/s40593-019-00186-y.

[6] M. J. Gierl and T. M. Haladyna, Eds., *Automatic Item Generation: Theory and Practice*. New York, NY, USA: Routledge, 2012.

[7] D. Liu, J. Wang, and L. Zheng, "Automatic test paper generation based on ant colony algorithm," *Journal of Software*, vol. 8, no. 10, pp. 2600–2606, Oct. 2013, doi: 10.4304/jsw.8.10.2600-2606.

> **Note on [1] and [2]:** both are standard textbooks confirmed to exist; the *edition and year* shown are the intended ones but should be checked against the physical copies before the final build (GUIDE §5 rule 3). They remain in the live list because the works themselves are certain — only the edition detail is pending confirmation.

Outstanding citation gaps (keep `{{CITATION NEEDED}}` in the chapter — do NOT invent)

These in-text markers have **no verified source** and must stay as `{{CITATION NEEDED}}` in the chapter until the user supplies or approves a real reference.

Chapter / section	Claim needing a source	Status / lead
`02-background.md` §2.1.2 (Blueprints & Test Specifications)	The assessment-theory notion of a *test blueprint / table of specifications* as a device for sampling intended content in intended proportions.	**No source in bank.** Needs a standard assessment/measurement reference. User must supply/approve — do not invent.
`02-background.md` §2.2	Related studies pursuing	**Quarantined candidate**

Chapter / section	Claim needing a source	Status / lead
(metaheuristic family)	test-paper generation "through refined genetic operators."	— see "Improved genetic algorithm" below. Confirm author list/year/pages before citing.
`02-background.md` §2.2 (integrated systems)	Integrated question-bank + paper-generation systems that package storage, selection, and assembly into one administrative tool.	**Quarantined candidate** — see "Adaptive question bank system" below. Confirm authors/pages before citing.

Candidates — real but details unconfirmed (resolve or drop before final)

Do **not** cite these in a chapter until their bibliographic details are confirmed (author list, year, pages). Held here, out of the live References list, so they aren't lost. If confirmed, each would be inserted at its first-appearance position in §2.2 and every later `[n]` renumbered accordingly.

- **Improved genetic algorithm for test-paper generation** — "Application of improved genetic algorithm in automatic test paper generation," IEEE Xplore doc. 7382551. *(Authors/year/pages unconfirmed — IEEE Xplore blocked automated fetch.)* Target slot: `02-background.md` §2.2, the "refined genetic operators" gap.
- **Adaptive question bank system** — "Designing an Adaptive Question Bank and Question Paper Generation Management System," Springer (LNNS), doi: 10.1007/978-981-15-3514-7_72, 2020. *(Authors/exact pages unconfirmed.)* Target slot: `02-background.md` §2.2, the "integrated systems" gap.

Bibliography (read but not cited)

(Sources consulted but not cited in the text — syllabus p. 106 permits a separate Bibliography. None at present: every verified source consulted is cited in the References list above.)

Appendices

The appendices collect supporting material that supplements the main report: screenshots of the running system, selected excerpts of the major source components, and the log of supervisor visits. The source excerpts in Appendix B are reproduced verbatim from QForge's own codebase as exhibits; each is labelled with the file it is taken from.

Appendix A — Screenshots

The figures below capture the principal user-facing flows of QForge, from authoring a blueprint to exporting a finished paper. Each figure is a screenshot of the running Vue frontend talking to the Laravel API. Images are inserted at print; the caption fixes each figure's meaning and number.

{{SCREENSHOT: report/figures/screenshots/a1-blueprint-editor.png}}

Figure A.1: Blueprint editor — defining sections, slots (question type and marks), allowed units, unit maximums, and the non-repetition window that the generator later compiles into constraints.

{{SCREENSHOT: report/figures/screenshots/a2-generate-result.png}}

Figure A.2: Generate-paper result — a previewed paper with its per-constraint pass/fail panel (section counts, total marks, unit coverage, unit maximums, no in-paper repetition) and the seed echoed for reproducibility.

{{SCREENSHOT: report/figures/screenshots/a3-infeasible-shortfall.png}}

Figure A.3: Infeasible blueprint — the best-effort partial paper with the named shortfall (missing slots and the scarcest units), offering AI bank expansion when the gap is one of supply.

{{SCREENSHOT: report/figures/screenshots/a4-question-bank.png}}

Figure A.4: Question bank — the approved question pool for a subject, filtered by unit, type, and marks.

{{SCREENSHOT: report/figures/screenshots/a5-review-queue.png}}

Figure A.5: Review queue — questions extracted from an uploaded past paper (or generated by the AI top-up) awaiting an admin's approve/reject decision before they enter the bank.

{{SCREENSHOT: report/figures/screenshots/a6-document-upload.png}}

Figure A.6: Document upload — a past paper or syllabus PDF submitted for extraction, showing the processing status (uploaded → processing → parsed/failed).

{{SCREENSHOT: report/figures/screenshots/a7-export.png}}

Figure A.7: Export — a saved paper rendered for PDF/DOCX download.

> Screenshots are `{{PLACEHOLDER}}` exhibits the author supplies at print. Add or drop figures to match the actual captured flows; keep captions numbered and in flow order.

Appendix B — Source Code Snippets

The following are **selected excerpts** — not whole files — of the major components, chosen to evidence the constraint-based generation algorithm that is the project's centrepiece. Each excerpt is headed by the class, method, and file it is taken from. The full sources live under `code/app/Services/PaperGeneration/` and `python-service/app/`.

B.1 — Generation facade: compile → greedy → validate → backtrack

From `PaperGenerator::generate()` in `code/app/Services/PaperGeneration/PaperGenerator.php`. The facade orchestrates the five phases and returns either a fully-valid paper or a best-effort partial with a named shortfall. It reads the bank only — persistence is the controller's job — which keeps the engine unit-testable in isolation.

```
public function generate(Blueprint $blueprint, ?int $seed = null, ?SimilarityGuard $guard = null): GenerationResult
{
                $guard ??= new NullSimilarityGuard;
            $compiled = $this->compiler->compile($blueprint);
```

```
      // Candidate-exclusion rules composed into one closure, applied to both pools.
                $exclusions = $this->composeExclusions([
                 $this->lastNExclusion($blueprint, $compiled),
               $this->examYearExclusion($blueprint, $compiled),
                                                              ]);

               // Pass 1 — greedy (LRU) fill.
     $greedy = $this->greedyFill($compiled, $exclusions, $seed, $guard);
          $constraints = $this->validator->validate($compiled, $greedy);

         if ($this->isComplete($compiled, $greedy) && $this->validator->allPass($constraints))                                {
           return new GenerationResult(true, $compiled, $greedy, $constraints,                                                  []);
                                                                 }

     // Pass 2 — backtracking repair, unless the blueprint is structurally                                          infeasible.
            $candidatesBySlot = $this->poolsBySlot($compiled, $exclusions);
             $resolved = $compiled->structurallyInfeasible()
                                                          ? null
         : $this->resolver->resolve($compiled, $candidatesBySlot, $seed);

                      if ($resolved !== null) {
            $constraints = $this->validator->validate($compiled, $resolved);

          return new GenerationResult(true, $compiled, $resolved, $constraints,                                               []);
                                                                 }
```

```
      // Infeasible — return the best-effort partial plus the shortfall.
      $missing = $this->computeMissingSlots($compiled, $greedy, $candidatesBySlot);

       return new GenerationResult(false, $compiled, $greedy, $constraints, $missing);
}
```

B.2 — Greedy selection: coverage-first, least-recently-used, seeded tie-break

From `GreedySelector::pick()` in `code/app/Services/PaperGeneration/GreedySelector.php`. The selector picks the best candidate for a slot: questions covering a still-uncovered allowed unit rank first, then lowest `used_count` (least-recently-used rotation), with ties broken by the seeded `TieBreaker` key.

```
public function pick(Collection $candidates, array $uncoveredUnitIds = [], ?int $seed = null): ?Question
{
                                    return $candidates
              ->sort(function (Question $a, Question $b) use ($uncoveredUnitIds, $seed) {
             // A multi-unit question ranks uncovered-first when ANY tagged unit is still uncovered.
                  $aRank = array_intersect($a->taggedUnitIds(), $uncoveredUnitIds) !== [] ? 0 : 1;
                  $bRank = array_intersect($b->taggedUnitIds(), $uncoveredUnitIds) !== [] ? 0 : 1;

                               return [$aRank, $a->used_count, TieBreaker::key($seed, (int) $a->id)]
                                   <=> [$bRank, $b->used_count, TieBreaker::key($seed, (int) $b->id)];
                                                                  })
```

```
                                                        ->first();
}
```

B.3 — Backtracking repair: bounded, deterministic depth-first search

From `BacktrackingResolver::search()` in `code/app/Services/PaperGeneration/BacktrackingResolver.php`. When the greedy pass produces an invalid paper, this resolver searches for a fully-valid assignment. At each slot it orders candidates uncovered-unit-first, prunes any candidate that would exceed a per-unit maximum, and recurses; the base case accepts only when every allowed unit is covered.

```
$ordered = $pool
       ->reject(fn (Question $q) => in_array($q->id, $usedIds,
true))
    // Prune cap-busting candidates: already at maximum can never
appear in a valid assignment.
     ->reject(function (Question $q) use ($blueprint, $unitUse) {
                  foreach ($q->taggedUnitIds() as $unitId) {
                        if (isset($blueprint->unitCaps[$unitId])
                  && ($unitUse[$unitId] ?? 0) + 1 > $blueprint-
>unitCaps[$unitId]) {
                                                    return true;
                                                    }
                                                    }

                                                return false;
                                                    })
     ->sort(function (Question $a, Question $b) use ($uncovered,
$seed) {
        $aRank = array_intersect($a->taggedUnitIds(), $uncovered)
!== [] ? 0 : 1;
        $bRank = array_intersect($b->taggedUnitIds(), $uncovered)
!== [] ? 0 : 1;

          return [$aRank, $a->used_count, TieBreaker::key($seed,
(int) $a->id)]
```

```
            <=> [$bRank, $b->used_count, TieBreaker::key($seed,
(int)                                                   $b->id)];
                                                               })
                                                      ->values();

foreach        ($ordered        as        $question)        {
                    $assigned[$slot->index]     =     $question;
    // ... update per-unit use, then recurse one slot deeper ...
    $result = $this->search($blueprint, $candidatesBySlot, $depth
+  1,  $assigned,  [...$usedIds,  $question->id],  $nextUnitUse,
$seed);

                      if     ($result     !==     null)     {
                                            return     $result;
                                                               }

               unset($assigned[$slot->index]);    //    backtrack
}

return null;
```

B.4 — Constraint validation: structured pass/fail per constraint

From `ConstraintValidator::validate()` in `code/app/Services/PaperGeneration/ConstraintValidator.php`. The validator scores a set of selections against the blueprint and returns one structured pass/fail line per constraint. The two excerpts below show the unit-coverage and no-repetition checks (a multi-unit question covers every allowed unit it is tagged with).

```
// Unit coverage (only when the blueprint restricts units).
if        (!        empty($blueprint->allowedUnitIds))        {
                    $coveredIds     =     collect($selections)
            ->flatMap(fn (Question $q) => $q->taggedUnitIds())
                                                    ->unique()
                        ->intersect($blueprint->allowedUnitIds);
          $expectedCount   =   count($blueprint->allowedUnitIds);
```

```
                    $results[]    =    new    ConstraintResult(
                                    label:    'Unit    coverage',
                                expected:   "{$expectedCount}   units",
                                got:   $coveredIds->count().'   units',
                            pass:  $coveredIds->count()  ===  $expectedCount,
                                                                    );
}

//    No    repeated    questions    within    the    paper.
$ids  =  collect($selections)->map(fn  (Question  $q)  =>  $q->id);
$repeats    =    $ids->count()    -    $ids->unique()->count();
$results[]          =          new          ConstraintResult(
                    label:     'No     repeated     questions',
                             expected:       '0       repeats',
                            got:       "{$repeats}       repeats",
                            pass:       $repeats       ===       0,
);
```

B.5 — Soft semantic-duplicate guard in the greedy fill

From `PaperGenerator::greedyFill()` in `code/app/Services/PaperGeneration/PaperGenerator.php`. The within-paper near-duplicate guard is a soft preference, not a hard constraint: if dropping near-duplicates would empty the pool, the full pool is kept, so semantic dedup never causes a shortfall. It also fails open when the guard has no signal (RAG off or index down).

```
// Prefer questions that don't semantically repeat one already
placed.
$guard->prime($pool);
if          ($selections          !==          [])          {
                              $fresh       =       $pool->reject(
         fn (Question $q) => $guard->tooSimilar($q, $selections)
                                                    )->values();

                            if      ($fresh->isNotEmpty())      {
                                    $pool     =     $fresh;
                                                                }
```

```
}

$uncovered = array_values(array_diff($compiled->allowedUnitIds, $covered));
$pick = $this->selector->pick($pool, $uncovered, $seed);
```

B.6 — Generate endpoint: seed, reproducibility, and the shortfall response

From `PaperController::generate()` in `code/app/Http/Controllers/Api/PaperController.php`. The API draws a fresh random seed per run (so regenerations and different teachers vary), or accepts a client-supplied seed to reproduce an exact paper. An infeasible result returns the partial paper plus a machine-readable shortfall.

```
// A fresh random seed per run varies the paper; a client may pass its own to reproduce one.
$seed = $validated['seed'] ?? random_int(0, PHP_INT_MAX);

$result = $this->generator->generate(
                                        $blueprint,
                                             $seed,
                                  app(VectorSimilarityGuard::class),
);

if (! $result->satisfiable) {
                                     return response()->json([
                                       'satisfiable' => false,
                                            'seed' => $seed,
                                'blueprint_id' => $blueprint->id,
                                   'expandable' => ! $result->coverageStructurallyInfeasible(),
         'shortfall_reason' => $result->coverageDeficitMessage(),
               'paper' => $this->previewPaperPayload($blueprint, $result),
                'missing_slots' => $result->missingSlotsArray(),
                             'constraint_results' => $result->constraintResultsArray(),
```

```
        ]);
}
```

B.7 — Python processor: stateless PDF extraction

From the `/extract` endpoint in `python-service/app/main.py`. The Python service is a stateless processor: Laravel owns the database and decides what to persist. A past paper yields question candidates; a syllabus yields courses and units for admin confirmation. Failures come back as `{status: "error"}` with a 200 so the calling job records the reason rather than retrying a bad file.

```
@app.post("/extract", response_model=Envelope[ExtractData])
def extract(request: ExtractRequest) -> Envelope[ExtractData]:
    try:
        path = pdf.resolve_shared_path(request.path)
        pages = pdf.extract_pages(path)
    except (pdf.UnsafePathError, pdf.UnreadableDocumentError) as exc:
        return Envelope.fail(str(exc))
    except Exception as exc:  # noqa: BLE001 - surface the reason, never 500 the job
        logger.exception("extraction failed for %s", request.path)
        return Envelope.fail(f"extraction failed: {exc}")

    is_past_paper = request.type == "past_paper"
    candidates = parser.parse_pages(pages) if is_past_paper else []
    courses = [] if is_past_paper else syllabus.parse_courses(pages)

    return Envelope.ok(ExtractData(pages=len(pages), candidates=candidates, courses=courses, ...))
```

B.8 — Python processor: AI generation with partial success

From the `/generate-questions` endpoint in `python-service/app/main.py`. The AI top-up is processing only — no retrieval, no database. The valid subset comes back in

`data` and every malformed item in `errors`; one bad item never discards the whole batch. Laravel validates each question against the slot, resolves the unit, and persists.

```
valid:           list[GeneratedQuestion]          =          []
errors:               list[str]               =              []

for       index,       item       in       enumerate(raw_items):
                                                          try:
           valid.append(GeneratedQuestion.model_validate(item))
                    except     ValidationError     as     exc:
          reason = "; ".join(e["msg"] for e in exc.errors()) or
"invalid                                                question"
                    errors.append(f"item   {index}:   {reason}")

# Partial success is still success: the caller reads `data` for
what       to       store       and       logs       `errors`.
return Envelope(status="success", data=valid, errors=errors)
```

B.9 — Queued document processing: Laravel orchestrates, Python processes

From `ProcessDocumentUpload::handle()` in `code/app/Jobs/ProcessDocumentUpload.php`. Heavy extraction runs on a queue. The job calls the Python `/extract` endpoint, imports the returned candidates through Laravel, and records the outcome on `document_uploads` — keeping Laravel the orchestrator and Python a stateless processor.

```
public function handle(PythonService $python, CandidateImporter
$importer):                                                void
{
            $upload   =   DocumentUpload::find($this->uploadId);

         if   ($upload   ===   null   ||   $upload->isTerminal())   {
          return; // Deleted, or already processed by an earlier
attempt.
                                                             }

                              $upload->update(['status'       =>
```

```
DocumentUpload::STATUS_PROCESSING,     'error'     =>     null]);

       $data  =  $python->extract($upload->pythonPath(),  $upload-
>type);
     $counts = $importer->import($upload, $data['candidates'] ??
[]);

                                                $upload->update([
                    'status'   =>   DocumentUpload::STATUS_PARSED,
                                          'error'     =>     null,
                'meta'  =>  array_merge($upload->meta  ??  [],  [
                          'pages'  =>  $data['pages']  ??  null,
                     'questions_created'  =>  $counts['created'],
                  'questions_duplicate'  =>  $counts['duplicate'],
                        'courses'  =>  $data['courses']  ??  [],
                                                                ]),
                                                                ]);
}
```

Appendix C — Log of Supervisor Visits

The syllabus requires a record of supervisor consultations across the project's lifetime. The table below is completed by the author from the project logbook: each row records a meeting date, the discussion or feedback given, and the action taken in response.

{{SUPERVISOR_VISIT_LOG}}

Date	Discussion / Feedback	Action taken
{{DATE}}	{{DISCUSSION / FEEDBACK}}	{{ACTION TAKEN}}
{{DATE}}	{{DISCUSSION / FEEDBACK}}	{{ACTION TAKEN}}
{{DATE}}	{{DISCUSSION / FEEDBACK}}	{{ACTION TAKEN}}

Date	Discussion / Feedback	Action taken
{{DATE}}	{{DISCUSSION / FEEDBACK}}	{{ACTION TAKEN}}
{{DATE}}	{{DISCUSSION / FEEDBACK}}	{{ACTION TAKEN}}

The visit log is a `{{PLACEHOLDER}}` the author fills from the real logbook. Dates and feedback are not invented here; add rows as needed to match the number of recorded consultations.